

Tempest Fauna Trail

Project Documentation — a weather-driven roguelike built with AI coding agents

Merlin Duty-Knez (Meduty)

Adam Ebner (AdamZ0904)

<https://github.com/Meduty/tempest-fauna-trail>

July 2026

Contents

1	Overview	5
1.1	Status	5
1.2	Quickstart	5
1.2.1	Requirements	5
1.2.2	Setup	5
1.2.3	Weather API key (optional, but recommended)	6
1.2.4	Run	6
1.2.5	Tests	6
1.3	Project Map	6
1.4	AI-Driven Development	7
1.5	Build (Flet packaging)	7
1.6	License	7
2	Architecture	8
2.1	What the game is	8
2.2	The layered architecture	8
2.3	The combat engine -- the heart of the system	9
2.3.1	Where each piece lives	10
2.3.2	The tick model	10
2.3.3	The ability / passive / status framework (T.20)	10
2.3.4	Statuses, DOT, and barriers	11
2.4	The two weather systems (T.2)	12
2.5	Content layer -- champions, enemies, bosses, abilities	12
2.6	Power & scaling model (T.18)	13
2.7	Route & encounter generation	13
2.8	Economy & shop (T.22)	13
2.9	Data models & run state (T.1)	14
2.10	API layer (T.6 + T.7)	14
2.11	UI & visualization (mostly planned)	14
2.12	Dev tooling -- how to drive the game without a UI	15
2.12.1	Playtest CLI (tools/playtest/, T.27) -- interactive inspection	15
2.12.2	Power simulation (tools/simulation/, T.25) -- balance benchmarking	15
2.13	Determinism doctrine (read this before touching combat or content)	16
2.14	"Where do I find X?" -- quick index	16
2.15	End-to-end: what happens in one fight	17
2.16	Reading order for a newcomer	17

3	Systems & Features	19
3.1	Combat -- engine & resolution	19
3.1.1	Entry point (the only one)	19
3.1.2	The pieces	20
3.1.3	Tick model	20
3.1.4	Damage pipeline	21
3.1.5	Result	22
3.1.6	Rendering	22
3.1.7	Replay / forward stepper / inspect-at-tick (T.37b, T.37c)	23
3.1.8	Invariants this system owns	23
3.1.9	File map	24
3.2	Effects -- hooks, modifiers, statuses, registries	24
3.2.1	The shape	24
3.2.2	EventBus	24
3.2.3	StatusGate -- how statuses disable actions	25
3.2.4	CombatContext -- the mutator API	25
3.2.5	Registries -- id -> handler	25
3.2.6	Invariants this system owns	25
3.2.7	File map	26
3.3	Weather -- Favor & Affinity Clash	26
3.3.1	1. Weather Favor -- <code>combat_modifier</code>	26
3.3.2	2. Affinity Clash -- <code>damage_modifier</code>	26
3.3.3	Why decoupled	27
3.3.4	Also here	27
3.3.5	File map	27
3.4	Formation -- enemy spawn placement	27
3.4.1	Entry	27
3.4.2	The input contract -- <code>FormationPiece</code>	27
3.4.3	Placement policy	28
3.4.4	File map	28
3.5	Encounter & board -- squad generation, bosses, map effects	28
3.5.1	Determinism -- every roll derives from the seed	29
3.5.2	Difficulty	29
3.5.3	Squad generation	29
3.5.4	Bosses	29
3.5.5	Board & map effects	29
3.5.6	File map	30
3.6	Stat scaling	30
3.6.1	Power curve	30
3.6.2	Three scaling classes (T.33, V.34)	30
3.6.3	Quantities: int except <code>attack_speed</code>	31
3.6.4	Speed-stat baseline parity (#39, V.35)	31
3.6.5	Speed axis -- 7 levels (T.33b)	31
3.6.6	Combat tie-break (V.34, fixes B.14)	31
3.6.7	Where it lives	31
3.7	Weather API -- fetch, cache, refresher	32
3.7.1	Client -- <code>api/weather.py</code>	32
3.7.2	Cache -- <code>api/cache.py</code>	32

3.7.3	Refresher -- <code>api/refresher.py</code>	32
3.7.4	Node weather lifecycle -- persisted on the Run (T.39, V.73)	32
3.7.5	Invariants this system owns	33
3.7.6	File map	33
3.8	Save / serialization	33
3.8.1	The model is the schema	34
3.8.2	Back-compat rules (the part that bites)	34
3.8.3	File I/O (<code>game/save.py</code> , T.14)	34
3.8.4	Invariant	35
3.8.5	File map	35
3.9	Items -- engine, components, combined, equip, reward drops	35
3.9.1	Package layout	35
3.9.2	Data structures	35
3.9.3	Prep equip seam (<code>game/inventory.py</code> , T.23b)	36
3.9.4	Equip pipeline (combat-side)	36
3.9.5	Item factories (@register_item)	37
3.9.6	REWARD-node drops (<code>encounter.py</code>)	38
3.9.7	Combined items (remaining 20 -- T.29b)	39
3.9.8	Emblems (6 -- <code>emblems.py</code> , T.29b)	40
3.9.9	Special items -- run-actions (<code>special.py</code> , T.29b)	40
3.10	Kit design conventions -- faults to watch	41
3.10.1	Identity & role	41
3.10.2	Coefficients & scaling	41
3.10.3	Retaliation, sustain, persistence	42
3.10.4	Process & verification	43
3.11	Rosters -- champions, enemies, bosses	43
3.11.1	Counts (verified; /check re-checks against code)	43
3.11.2	Source of truth (code, not this doc)	43
3.11.3	Role taxonomy	44
3.11.4	Stat-axis balance (T.35b, #42 Finding B / B.20)	44
3.11.5	Axis-distribution rebalance (T.36)	44
3.11.6	Invariants	45
3.11.7	File map	45
3.12	Abilities & passives -- id resolution	45
3.12.1	Id convention	45
3.12.2	Registration	45
3.12.3	Counts (verified; /check re-checks)	46
3.12.4	The resolution guarantee (CI-guarded)	46
3.12.5	Ability descriptions (T.34)	46
3.12.6	File map	47
3.13	Traits -- kinships, callings, breakpoints	47
3.13.1	Where it lives	48
3.13.2	The 25 traits	48
3.13.3	Resolution (in <code>compile_loadout</code> , step 3)	48
3.13.4	Breakpoint shapes (<code>apex = min(pool, cap)</code> , V.37)	49
3.13.5	Combat primitives (T.28b) -- <code>game/traits/mechanics.py</code> + engine	49
3.13.6	Mechanic + apex riders (T.28c) -- <code>game/traits/mechanics.py</code>	50
3.13.7	Apex riders + arms (T.28d) -- <code>game/traits/mechanics.py</code>	51

3.13.8	Roster (post-T.28a rebalance, B.9)	51
3.13.9	Descriptions (T.41b) -- <code>game/traits/meta.py</code> + <code>game/describe.py</code>	52
3.13.10	Invariants	52
3.14	Items	52
3.14.1	Descriptions (T.41a) -- <code>game/items/meta.py</code> + <code>game/describe.py</code>	53
3.15	Augments -- run-long modifiers	53
3.15.1	Model	53
3.15.2	Combat seam (V.2 amendment, V.18)	53
3.15.3	Offers, reroll, quality curve	54
3.15.4	Quest trackers (§9.3)	54
3.15.5	Known simplifications (flagged, follow-ups)	54
4	AI-Agent Collaboration	55
4.1	The working model	55
4.2	The recurring lessons	55
4.2.1	1. Design docs lie -- verify every primitive against the code	55
4.2.2	2. Self-review is a separate, load-bearing step -- green tests are not "done"	56
4.2.3	3. Distrust the measurement -- a metric reading zero is a claim about the metric	56
4.2.4	4. Every bug becomes an invariant -- the backprop reflex	57
4.2.5	5. The human catches the hacks and redirects the method	57
4.2.6	6. Ask after a cheap investigation, not instead of one	57
4.2.7	7. Headless UI: the operator is the visual oracle -- until it isn't	57
4.3	How the collaboration evolved	58
5	How the Project Was Built	59
5.1	Spec-Driven Development	59
5.1.1	The document layers and the LIVING vs FROZEN rule	59
5.1.2	The skill chain	59
5.1.3	Invariants as executable guardrails	60
5.2	The implementation arc	60
5.2.1	Foundation (the deterministic core)	60
5.2.2	Framework and tooling	61
5.2.3	Content deepening and balance	61
5.2.4	The player-facing UI (the largest remaining build)	61
5.3	Determinism as the spine	61

Chapter 1

Overview

Weather-driven roguelike where animal champions battle across real-world cities. Live OpenWeather data shapes every fight -- a Snow-affinity team dominates in snowy Tokyo but folds when the city turns Thunder.

Built with [Flet](#) (Python), cross-platform desktop + web. Designed to be developed collaboratively with AI coding agents -- see [CLAUDE.md](#) and [SPEC.md](#).

1.1 Status

Engine + content + simulation layers are largely complete; the player-facing Flet UI is the main remaining build.

- [done] **Done** -- data models, tick combat engine (ability/passive/status framework, bosses, map effects), weather effects, 50-city route, champion/enemy/boss rosters + ability catalog, power scaling, encounter generation, economy & shop, OpenWeather client + cache + 3-stream refresher, theme/components, playtest CLI, power-simulation tooling.
- [planned] **Planned** -- most Flet views (menu, trail, prep, combat, summary), route-map + run-summary visualizations, save/load. See [SPEC.md §T](#) for per-task status.

For how the systems fit together and where to find them, read [ARCHITECTURE.md](#).

1.2 Quickstart

1.2.1 Requirements

- Python ≥ 3.10
- OpenWeather API key (free tier -- openweathermap.org/api)

1.2.2 Setup

```
git clone <repo>
cd tempest-fauna-trail
uv sync                                # or: pip install -e . && pip install -r requirements.txt
cp .env.example .env
# edit .env, set OPENWEATHER_API_KEY=<your-key>
```

1.2.3 Weather API key (optional, but recommended)

The Trail shows live weather per city, so it wants an [OpenWeather](#) key (free tier is plenty). Display only -- combat resolves on the node's deterministic default weather (V.66), so outcomes never depend on the live feed. Configure the key **either** way:

1. **.env file** (preferred for local dev) -- `cp .env.example .env`, then edit the one line:

```
OPENWEATHER_API_KEY=your_actual_key_here
```

No quotes, no spaces around =. `.env` is git-ignored; the app/tests load it via `python-dotenv`.

2. **Environment variable** (CI / shells / one-off):

```
export OPENWEATHER_API_KEY=your_actual_key_here
uv run flet run
```

3. **In-app Settings menu** (no terminal needed) -- launch the app, open **Settings** from the main menu, paste your key, **Save**. It persists to a local config file (`~/<user-data>/tempest-fauna-trail/config.json`) and the Trail picks it up next time you open it. An `OPENWEATHER_API_KEY` env var, if set, overrides it.

Without a key the game still runs -- the weather refresher simply never starts, so every Trail node shows a ? **"weather pending"** marker (the app **never** invents fake weather it hasn't fetched). Add a key to see live weather and the affinity favor/clash it drives. The key is read from the environment and **never logged** (V.3).

1.2.4 Run

```
uv run flet run          # desktop
uv run flet run --web    # browser
```

Note: the Flet entry point is still a placeholder shell (the game views are in progress -- see Status). To actually exercise the game today, use the headless **playtest CLI**, which drives the complete engine without a UI:

```
uv run python -m tools.playtest.sim_fight --help    # resolve one fight
uv run python -m tools.playtest.sim_run --help      # simulate a full run
```

1.2.5 Tests

```
uv run pytest          # all tests
uv run pytest -m "not integration"  # unit only (no network)
uv run pytest -m integration      # live API tests (needs OPENWEATHER_API_KEY)
```

1.3 Project Map

Path	Purpose
SPEC.md	Canonical spec -- goals, invariants, tasks, bugs. Source of truth.
ARCHITECTURE.md	System map -- how the engine/content/weather/economy systems work and where each lives in the code
CLAUDE.md	AI agent onboarding -- project structure, conventions, workflow
docs/proposal.md	Pitch + design framing

Path	Purpose
docs/design/tasks/	Per-task plan docs (<code>tn*_plan.md</code>)
docs/design/content/	Champion / enemy / boss / augment / item / trait rosters
docs/design/systems/	Combat, passive, effect, view system designs
docs/journal/	Chronological dev log
src/	Implementation
tests/	Unit + integration tests

1.4 AI-Driven Development

This repo is set up for collaboration with AI coding agents (Claude Code etc.):

- **Source of truth** lives in [SPEC.md](#) -- caveman-encoded sections §G/§C/§I/§V (invariants), §T (tasks), §B (bugs), §D (deferred).
- **Skills** wrap key workflows:
 - `/spec` -- sole mutator of `SPEC.md` (new spec, amend, distill from code, bug backprop)
 - `/build` -- plan-then-execute against §T tasks; auto-runs backprop on test failure
 - `/check` -- read-only drift audit, SPEC vs code
- **Per-path guardrails** in [.claude/rules/](#) bound AI edits by scope (`api.md`, `game-logic.md`, `flet-ui.md`).
- **Journal** in [docs/journal/](#) captures the "why" that SPEC compresses out -- append after non-trivial milestones.

1.5 Build (Flet packaging)

```
flet build apk -v           # Android
flet build ipa -v          # iOS
flet build macos -v
flet build linux -v
flet build windows -v
flet build web -v
```

See [Flet build docs](#) for signing + distribution details.

1.6 License

TBD.

Chapter 2

Architecture

How Tempest Fauna Trail is built, how the systems fit together, and **where to find each one in the code**. This is the map for understanding the game and finding your way around the repo.

How the docs relate. [SPEC.md](#) is the *contract* (goals, invariants §V, task table §T, bug log §B) -- the source of truth for *what must hold*. [CLAUDE.md](#) is the *agent quick-start*. The [journal](#) is the *narrative* -- why decisions were made. **This file is the *system map*** -- what exists, how it interacts, and the file you open to touch it. When in doubt about a rule, SPEC wins; when lost in the tree, start here.

2.1 What the game is

A roguelike **auto-battler** (TFT-inspired) themed around animal spirits and weather. You pick **1 champion**, start with **10 Amber**, and travel a fixed **50-node route** of real-world cities across 6 continent stages. **Live OpenWeather data** at each city warps combat. Battles are **tick-based and auto-resolved** -- all player agency happens *between* fights.

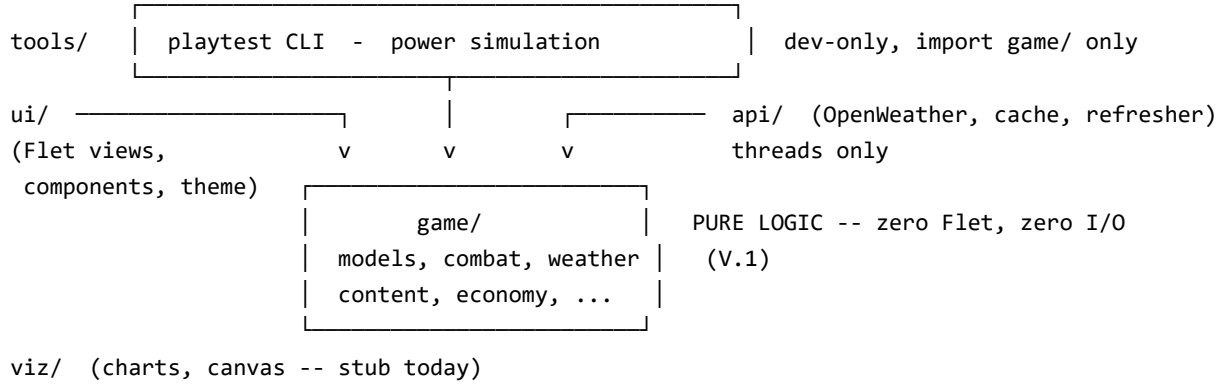
The per-node loop (SPEC §G):

```
Trail  —> Prep  —> Combat  —> (rewards: Amber / items / Tempest XP) —> next node
view route  shop, board  auto-resolved
+ preview  placement,    tick battle
weather    level team
```

The player's decisions: **team composition + synergy traits, item optimization, weather-aware roster swaps, board positioning**. The engine then resolves the fight deterministically.

2.2 The layered architecture

Four source layers under `src/`, plus dev tooling under `tools/`. The **cardinal rule is dependency direction** -- everything points *inward* toward pure game logic:



Invariants that enforce this (SPEC §V):

Rule	Invariant
game/ has zero Flet imports -- pure logic	V.1
tools/simulation/ imports only from src/game/ (no ui/, no api/)	V.14
All HTTP runs on a worker <code>threading.Thread</code> , never the main thread	V.4
API failure never crashes -- leaves a node unknown/substitute, keeps retrying	V.3

Because `game/` is pure and I/O-free, the **entire game is testable and simulatable without a UI or network** -- which is exactly what `tools/` exploits.

2.3 The combat engine -- the heart of the system

Combat is a **single pure function** (V.2):

```
resolve_combat(team, enemies, weather, *, node_id="") -> BattleResult
```

Identical inputs -> byte-identical output. No RNG, no clock, no globals. (SPEC V.2 also specifies an optional `run_mods: RunModifiers` arg for active augments -- that's the planned **T.31** extension and is *not yet in the code*; today's signature is the one above.) It internally delegates through a fixed chain:

```

compile_loadout(team, enemies, weather)      # content <-> combat boundary
    | -> (pieces, EventBus)
    v
CombatContext(pieces, bus, weather, seed)    # the mutator API (board_state optional)
    |
    v
combat/engine.run(ctx)                       # the ONE tick loop (V.29)
    | events -> EventBus
    v
BattleResultRecorder.build_result()          # rebuilds BattleResult from events
  
```

Boss fights insert one extra step -- `attach_map_effect(...)` after building the context and before `run()`. The canonical wiring is `src/game/combat/resolve.py::resolve_boss_combat` (T.12b/V.59 -

- the single src-side boss entry, takes a `map_effect_id: str` so `combat/` stays content-import-free; `tools/playtest/_common` delegates to it). `CombatReplay/` `inspect_at_tick` accept the same `map_effect_id` to replay a boss fight.

2.3.1 Where each piece lives

Concern	File
Public entry (<code>resolve_combat</code>) + the shared <code>build_combat</code> wiring helper (compile -> <code>assign_spawns</code> -> context, optional recorder) reused by <code>resolve</code> / <code>boss</code> / <code>replay</code>	<code>src/game/combat/resolve.py</code>
Package re-exports (<code>resolve_combat</code> , <code>CombatContext</code> , <code>run</code> , <code>inspect_at_tick</code> , <code>CombatReplay</code>)	<code>src/game/combat/__init__.py</code>
The tick loop -- the single <code>_step_combat</code> generator (energy meters, pathing, attacks, casts, statuses, map effects, sudden death) + tuning constants; <code>run</code> drains it, <code>CombatReplay</code> steps it forward (T.37c)	<code>src/game/combat/engine.py</code>
Mutator API -- the <i>only</i> way content touches the world	<code>src/game/combat/context.py</code>
Event -> <code>BattleResult</code> reconstruction (beats + <code>initial_pieces</code> board snapshot)	<code>src/game/combat/recorder.py</code>
Replay -- <code>CombatReplay</code> steps the engine forward for playback; <code>inspect_at_tick</code> re-runs to a tick (random seek) on a cloned <code>run_mods</code> ; both return read-only <code>PieceViews</code> , record nothing (V.55); the live state is the view's resource truth, not the event stream (V.56/V.57, B.28)	<code>src/game/combat/replay.py</code>
Compile models -> combat <code>Pieces</code> + wire passives/weather/ traits	<code>src/game/loadout.py</code>
Synergy traits -- <code>TraitScope</code> / <code>TraitBreakpoint</code> / <code>DynamicThreshold</code> , <code>@register_trait</code> , <code>_resolve_traits</code> (unique-id count, affinity synthesis, apex/dynamic threshold) applied in <code>compile_loadout</code> step 3 (T.28a; primitives T.28b/c)	<code>src/game/traits/</code>

V.29 -- there is exactly one tick loop. `engine.py` is it. The old `loop.py` was deleted after the T.26 unification; do not reintroduce a parallel engine (see the [2026-06-04 journal](#)).

2.3.2 The tick model

Time is discretized into **10ms ticks**. A piece acts when its **action energy meter** overflows `ENERGY_THRESHOLD`; movement uses a separate meter. One action `~= **~600 ticks (~6s)**` -- this scale matters: it's why DOT fires on a *per-status cadence*, not per tick (V.25). Meter overflow carries over, and triggered meters resolve in a **deterministic order** so replays stay identical.

2.3.3 The ability / passive / status framework (T.20)

Content (abilities, passives, items, traits, augments) never mutates combat state directly. It plugs in through three declarative primitives, then reacts through events:

- **EffectBundle** (effects.py) -- a bag of Modifiers, Hooks, granted statuses / abilities / traits. The unit of "what this thing does."
- **Modifier** (effects.py) -- a stat change with a Lifetime (PERMANENT / COMBAT / TIMED). Stats are computed via compute_stat layering base + modifiers.
- **Hook** (effects.py) -- a callback subscribed to the **EventBus** by event name, with a HookScope dedup (PER_HIT / ONCE_PER_CAST / ONCE_PER_TARGET / ONCE_PER_COMBAT). Real event names the loop emits include on_attack_landed, on_attack_start, on_damage_pre / on_damage_dealt / on_damage_taken, on_cast / on_cast_complete, on_death, on_kill, on_heal, on_tick, on_status_applied / on_status_expired, on_spawn, on_combat_start / on_combat_end. Typed payloads live in events.py (AttackEvent, DamageEvent, ...).
- **Registries** (registries.py) -- ABILITY_REGISTRY (144) + PASSIVE_REGISTRY (147)
 - TRAIT_REGISTRY (25, T.28+Multicaster) + ITEM_REGISTRY (50, T.29a-d)
 - RUN_ACTION_REGISTRY (5, T.29b) populated; AUGMENT_REGISTRY is still an empty scaffold awaiting its content (T.31). Content factories self-register via @register_* decorators; importing the content package triggers them. Lookups are by **string id**.
- **Presentation layer** (registries.py + ability_text.py, T.34/T.35) -- a parallel ABILITY_META (285 ids) gives every roster ability a tooltip. Numeric outlets flow through the **closed Magnitude family** (ScalingTerm linear / PctResource / MaxOfTerm / SetByCaller, GAS-modeled, V.46): the handler reads the number via term.eval(...) and ability_text.render renders the *same* object (source-of-truth B, V.38 -- tooltip can't drift from combat). Pure, no Flet (V.1). An AST guard (test_no_orphan_stat_reads) fails the build on any handler stat-read not backed by a Magnitude.
- **Description render-layer** (game/describe.py + items/meta.py + traits/meta.py, T.41) -- the same pattern for **champion-independent** content (items + traits): RenderedEntry / RenderedTrait (name + blurb + stat line) from a per-domain *_META (authored name/blurb, transcribed from the design catalogs; trait rung counts reconciled to code, V.79). The **stat line is derived from the live numbers** -- introspected from the item's EffectBundle (V.78) or from traits/_packs.TRAIT_STAT_PACKS (V.79), never re-typed, so it can't drift; no Magnitude machinery (these grant fixed %, not caster-scaled). Pure, no Flet, no mutation (V.80). Consumed by the Prep item chips + the trait_synergies_panel tooltips (Prep + Combat).

V.15 / V.22 / V.17 / V.38 / V.46 / V.47 -- every id resolves + every scaler is visible. Any ability_id, passive_id, trait tag, or augment id referenced by content data must resolve in its registry; every ability id must have an ABILITY_META; every handler stat read must be a Magnitude; an int/hybrid unit must read INT in its kit. All CI-guarded.

The **CombatContext** (context.py) is the single mutation point: ctx.deal_damage, ctx.apply_status, ctx.grant_barrier, ctx.heal, ctx.register_bundle, etc. "Direct mutation architecture" -- no effect-as-data reducer.

2.3.4 Statuses, DOT, and barriers

- **Statuses** (status.py, piece.py) -- id-based, **one StatusInstance per status_id per piece** (V.26, non-stacking identity; intensity that should accumulate uses StackBehaviour.STACK, e.g. poison). Carry gates (stun -> can't act), DOT info, and stacks.
- **DOT cadence** (V.25) -- DOT damage + stack decay fire on a **per-status clock** (dot_interval_ticks, default 100t = 1s), *not* every engine tick. The clock free-runs (re-applying

never resets the next tick). Stack decay is **percentage** (`decay_fraction`), giving an investment-scaling equilibrium with **no hard cap** -- see [journal](#) and the *no-hard-caps* balance philosophy.

- **Barriers** (V.28, `piece.py`) -- a temporary absorb pool **distinct from HP and from "shield"**. Soaked before HP inside `deal_damage`, FIFO, optional tick expiry, granted only via `ctx.grant_barrier`. "Shield" in content ids means an armor/resistance *buff*, a different mechanic -- do not conflate.

2.4 The two weather systems (T.2)

`src/game/weather_effects.py` -- two decoupled systems, never summed:

1. **Weather Favor** (`combat_modifier`) -- does the *node weather* suit my affinity? A 5-tier stat buff/debuff applied **once at combat init**, in the single application path `compile_loadout::_apply_weather_to_piece`.
2. **Affinity Clash** (`damage_modifier`) -- do *I* beat *this enemy*? A per-hit damage multiplier by attacker-affinity vs defender-affinity, resolved **per hit** in the loop (it depends on the defender, so it can't be pre-snapshotted).

Both run off the **single affinity: WeatherState field** every piece carries (V.6 -- there is no separate weakness field). CLEAR sits outside the predator/prey ring and is inert in both. The 6 weather states (V.5) map 1:1 to OpenWeather id groups.

2.5 Content layer -- champions, enemies, bosses, abilities

Concern	File
Champion / enemy rosters + base-stat template + level scaling	<code>src/game/content.py</code>
Champion ability/passive handlers (~60 champions)	<code>src/game/abilities/champions.py</code>
Enemy ability/passive handlers (~60 enemies)	<code>src/game/abilities/enemies.py</code>
Boss kits -- 6 two-phase bosses	<code>src/game/abilities/bosses.py</code>
Boss definitions, supporting cast, phase/death hooks	<code>src/game/bosses/data.py</code>
Boss map effects (decoupled board hazards)	<code>src/game/map_effects.py</code> , <code>src/game/board.py</code>
Ability tooltips -- AbilityMeta + Magnitude family + renderer (T.34/T.35)	<code>src/game/registries.py</code> , <code>src/game/ability_text.py</code>
Synergy traits (Kinship / Calling / Affinity)	<code>docs/design/content/trait_catalog.md</code> -> <code>game/traits/</code> (T.28 [done])
Items	<code>game/items/</code> (T.29a-d [done] -- components, combined, emblems, special run-actions, mana primitive, multi-slot)
Augments	<code>game/augments.py</code> (T.31 [planned] -- planned)

Content **vocabulary lives with content** (V.8): synergy tags are open-ended strings the engine treats as opaque labels. The roster has a history of drifting from the catalog docs (e.g. `CALLING_TAGS` once carried dead tags) -- always diff code vocabulary against `docs/design/content/*_catalog.md` and add a V-guard (see `CLAUDE.md` planning rules).

2.6 Power & scaling model (T.18)

`src/game/scaling.py` -- one formula governs all power:

$P(T, L) = 2^{((T-1)/3 + \text{triplings}(L))}$ `triplings = {L1:0, L2:1, L3:3}`

Stat multiplier is $\sqrt{P}$ so that HP-DPS ($\sim$ combat value) grows *linearly* with P, keeping encounter budgets linear. Level-ups use a **tripling** mechanic (3 copies -> next level), giving an accelerating curve (L2 modest, L3 a spike). Total T1L1->T10L3 $\sim$ **8x in stats**. Tier-10 "Primordials" are boss-only -- the buyable ceiling is T9.

2.7 Route & encounter generation

Concern	File
Fixed 50-node route, 6 stages, city coords + enemy pools	<code>src/game/route.py</code>
Seed-deterministic squads for FIGHT/REWARD/CHALLENGE/BOSS	<code>src/game/encounter.py</code>
Role-aware enemy board placement	<code>src/game/formation.py</code>

Determinism doctrine (V.19 + the T.19 contract): all "randomness" derives from (`run_seed`, `node_index`, `channel`) -- no clock, no global RNG. Same seed -> same route draws, same squads, same shop, same economy. This is what makes the simulation layer trustworthy. Enemy formation (`formation.py`) is fully deterministic and role-aware (tanks col 7, bruisers col 8, backline col 9, bosses authored positions).

2.8 Economy & shop (T.22)

Concern	File
Amber income, interest, Tempest team-size progression	<code>src/game/economy.py</code>
Stage-gated tier rolls, buy / sell / reroll	<code>src/game/shop.py</code>

Currency is **Amber**; team-size XP is **Tempest** (rank = deployable board cap, 1->10, **monotonic non-decreasing**, V.20). Shop offers are seed-deterministic (`run_seed`, `visit_index`, `reroll_count`) (V.19). These are the **headless economy substrate** the (planned) Prep UI will drive.

2.9 Data models & run state (T.1)

src/game/models.py -- all dataclasses (slots=True), JSON-serializable:

- WeatherState, NodeType, NodeState, RunStatus, CombatOutcome (enums)
- Champion, Enemy -- roster/source models (carry affinity, traits, stats, ability ids)
- Node -- one route stop
- BattleEvent, BattleResult -- combat output + event stream (the runtime combat entity is Piece in piece.py, built by the loadout compiler)
- Run -- the single object holding *all* game state (current node, roster, battle log, Amber, Tempest, augments). Per V: one Run is the whole game.

src/game/save.py (T.14) -- the **file-I/O layer** over that contract: save_run (atomic temp+os.replace), load_run (schema_version gate -> Run.from_dict), default_save_dir, CURRENT_SCHEMA_VERSION, and typed errors (SaveError / CorruptSaveError / UnsupportedSchemaError). No Flet import (V.1); the (de)serialization contract itself stays on the dataclasses. See [docs/live/systems/save.md](https://docs.live/systems/save.md).

2.10 API layer (T.6 + T.7)

Concern	File
OpenWeather client -- fetch by lat/lon, parse -> WeatherState	src/api/weather.py
Stateless per-city cache (unknown / live+fetches_at / substitute)	src/api/cache.py
3-stream tick refresher (<=3 calls/min)	src/api/refresher.py

The refresher ticks 1/min and fires three deduped streams (A: full round-robin 50; B: window [current+1..+6]; C: uniform random) -> bounded API usage, staleness <= 50 min (V.11). On fetch failure a node falls back to the city's default_weather flagged substitute (V.3, V.13). Key via OPENWEATHER_API_KEY, **never logged** (V.3). All HTTP on a worker thread (V.4).

Persistence (T.39, V.73). The cache is the fetch-scheduling/freshness layer; the **persisted source of truth is the Run Node**: each node carries weather (the effective game weather - default_weather placeholder until a live fetch overwrites it), weather_state: NodeWeatherState {unknown/live/substitute}, and weather_locked. The Trail writes fetched cache values onto the Run via the pure game-side mutators Run.set_node_live_weather / Run.lock_node_weather only (cache/refresher stay stateless re: game, V.10). The **current** node's weather **locks at the Trail->Prep transition** (frozen for the run; refresher skips locked nodes) -- so live weather reaches combat (Weather Favor, CHALLENGE rolls) while staying byte-identical across the fight + reward (load-bearing for V.70).

2.11 UI & visualization (mostly planned)

Concern	File	Status
Design tokens (colors, type, spacing, animation)	src/ui/theme.py	[done] T.8
Shared components (champion card, weather badge, meter bar, chips)	src/ui/components/	[done] T.8
Playtest admin panel (dev)	src/ui/views/admin.py	dev tool
Flet entry point -- menu app shell + Playfight harness + admin	src/main.py	[done] T.9
Main menu (/) -- New Run (live) / Continue (disabled until load-into-Trail, T.15) / Playfight / Quit	src/ui/views/menu.py	[done] T.9
RunStart (/run-start) -- seed-deterministic 1-of-3 champion pick -> Run	src/game/run_init.py, src/ui/views/run_start.py	[done] T.10
Combat view + dev/Playfight harness	src/ui/views/combat.py, dev_harness.py	[done] T.12
Trail / Prep / Reward / Summary views	src/ui/views/	[partial] T.11-T.15/T.23
Route map (Canvas), run summary (BarChart)	src/viz/	[planned] T.11, T.13 (stub)

The game logic is complete; the player-facing Flet UI is the largest remaining build. `main.py` is the menu-rooted `page.views` shell (T.9): **Playfight** opens the combat dev harness -> combat view; **New Run** opens `RunStart` (T.10, `game/run_init.new_run`) -> champion pick -> Run (currently lands on a Trail stub until T.11); **Continue** waits on load-into-Trail (T.15); `TEMPEST_ADMIN=1` opens the admin panel and `TEMPEST_DEV=1` jumps straight to Playfight. See [SPEC §I Flet Routes](#) and `views_spec.md`. Flet conventions live in [.claude/rules/flet-ui.md](#) and `CLAUDE.md`.

2.12 Dev tooling -- how to drive the game without a UI

Because `game/` is pure, two toolkits exercise it directly:

2.12.1 Playtest CLI (`tools/playtest/`, T.27) -- interactive inspection

```
uv run python -m tools.playtest.sim_fight --help      # one fight
uv run python -m tools.playtest.sim_node ...          # a node encounter
uv run python -m tools.playtest.sim_run ...           # a full run
uv run python -m tools.playtest.inspect ...           # roster inspection
uv run python -m tools.playtest.inspect_node ...
```

`_common.py` holds shared helpers; `resolve_boss_combat` now delegates to the src-side `combat/resolve.py` entry (V.59).

2.12.2 Power simulation (`tools/simulation/`, T.25) -- balance benchmarking

- `matchup.py` -- `run_matchup`, the pure unit of work (safe in worker processes)

- `tournament.py` -- battle generators (1v1, team2, sampled teams)
- `runner.py` -- CLI entry (`python -m tools.simulation.runner --mode 1v1 ...`)
- `mega.py` -- runs every sweep flavour in one parallelized go
- `ratings.py` -- win-rate + power-model metrics, incl. cross-weather `weather_metrics`
- `report.py` -- CSV / console output

Sim metric invariants: weather-affinity metrics are **cross-weather** (V.16) and **treat absent weather as NaN, never 0** (V.30) -- averaging missing-as-0 once fabricated a balance signal (B.12). Sim outputs land in `results/` (gitignored); published analyses live in `reviews/`.

2.13 Determinism doctrine (read this before touching combat or content)

Determinism is non-negotiable (V.2, V.14, V.19). The simulation and replay systems depend on it:

- **No RNG in mechanics.** Any "chance" / "every few hits" effect uses a **deterministic cadence counter** (like `crit_counter`), never random.
- **All procedural generation** derives from (`run_seed`, `node_index`, `channel`).
- **resolve_combat is pure** -- same inputs, byte-identical `BattleResult`. When the T.31 `run_mods` arg lands it must default to `None`, leaving every existing caller (and every sim) byte-for-byte unchanged (V.2).
- **State for a view is recomputed, not recorded (V.55).** The `CombatReplay` forward stepper (playback) and `inspect_at_tick` (random seek) drive the same single `_step_combat` generator to read any piece's live state at any tick -- no per-tick keyframes in `BattleResult`. Same purity contract: they clone `run_mods` so the replay can't mutate the caller's quest state. This live state is the combat view's resource truth, **not** the event stream's partial `hp_after` (V.56/V.57, B.28).
- **Verify with:** fixed seed + `workers=1` -> identical output across runs.

2.14 "Where do I find X?" -- quick index

I want to...	Go to
Change how a fight resolves	<code>src/game/combat/engine.py</code>
Add/modify what content can do to the world	<code>src/game/combat/context.py</code> (mutator API)
Add a champion/enemy ability	<code>src/game/abilities/{champions,enemies}.py</code> + register
Add a boss	<code>src/game/bosses/data.py</code> + <code>src/game/abilities/bosses.py</code> + <code>src/game/map_effects.py</code>
Tune stats / scaling	<code>src/game/content.py</code> , <code>src/game/scaling.py</code>
Change weather effects	<code>src/game/weather_effects.py</code>
Edit the route / cities	<code>src/game/route.py</code>
Change enemy generation	<code>src/game/encounter.py</code> , <code>src/game/formation.py</code>

I want to...	Go to
Touch the economy / shop	<code>src/game/economy.py</code> , <code>src/game/shop.py</code>
Add a data field to game state	<code>src/game/models.py</code> (Run, Champion, ...)
Change weather fetching / caching	<code>src/api/{weather,cache,refresher}.py</code>
Build a UI screen	<code>src/ui/views/</code> (+ <code>theme.py</code> , <code>components/</code>)
Balance-test a change	<code>tools/simulation/</code>
Manually try a fight/run	<code>tools/playtest/</code>
Understand <i>why</i> a decision was made	<code>docs/journal/</code>
Know what rule must hold	SPEC.md §V

2.15 End-to-end: what happens in one fight

1. Player advances to a node; `encounter.py` generates the enemy squad from the seed; `formation.py` places them; the node's weather is read from the persisted Node (`node.weather` -- live values written through from the cache while on the Trail, then **frozen by the Pre-entry lock**, T.39/V.73).
2. `resolve_combat(team, enemies, weather)` is called.
3. `compile_loadout` builds runtime Pieces, applies **Weather Favor** to base stats, subscribes passive Hooks to a fresh EventBus, wires boss phase/death hooks.
4. `CombatContext` wraps the pieces + bus + board; bosses `attach_map_effect`.
5. `engine.run(ctx)` ticks: meters fill -> pieces move (BFS pathing) / auto-attack / cast abilities; every damage instance applies **Affinity Clash**; statuses tick on their cadence; barriers soak; map effects fire; sudden-death timeout guards stalls. Every action emits a typed event onto the bus.
6. `BattleResultRecorder` (subscribed to the bus) reconstructs a `BattleResult` -- outcome, survivors, the full beat stream (move/attack/cast/death + heal/dot/ status/spawn/despawn, each beat one event with `hp_after/barrier_after` for HP-changing ones, T.37a) and an `initial_pieces` board snapshot + board dims so a combat view can lay out and animate the fight without re-deriving formation.
7. The result flows to `Run.battle_log`; rewards (Amber / items / Tempest) are applied; the next node opens.

2.16 Reading order for a newcomer

1. **This file** -- the map.
2. [SPEC.md](#) §G (goal), §V (invariants) -- the rules.
3. `src/game/models.py` -- the vocabulary.
4. `src/game/combat/engine.py` + `context.py` -- the engine.
5. `src/game/abilities/champions.py` -- see content plug into the framework.
6. Run `tools/playtest/sim_fight` -- watch a fight resolve.
7. [docs/journal/](#) -- the "why" behind the non-obvious choices.

For per-system design depth, see the [Documentation Map in CLAUDE.md](#) and `docs/design/`. For the status of any task, see [SPEC.md §T](#).

Chapter 3

Systems & Features

This chapter is the mid-level reference layer: how each subsystem and content area works *now*, and where it lives in the code. It is assembled from the project's **LIVING** documentation (`docs/live/`) -- docs that are reconciled to the code on every change that touches their subject and audited by the `/check` drift tool. Where the previous chapter mapped *what exists and how it interacts*, this one goes one level deeper into each system in turn.

The sections below cover, in order, the combat engine and its resolution pipeline, the effect/hook/status framework that all content plugs into, the two weather systems, enemy formation, encounter and board generation, stat scaling, the weather API layer, save/serialization, items, and the kit-authoring conventions -- followed by the content references (rosters, abilities, traits, augments, items) whose real data lives in code.

3.1 Combat -- engine & resolution

Status: **LIVING** -- must match `src/game/combat/`, `src/game/loadout.py`, `src/game/piece.py`. Audited by `/check`. **Scope:** how one battle resolves end-to-end: entry -> compile -> tick loop -> result. **Reconciled:** 2026-06-05 @ refactor/combat-engine-single-source.

Citations are by **symbol** (file + function), not line number, on purpose -- lines drift, symbols don't. Design rationale lives in the frozen `docs/design/systems/combat_system_proposal.md` + `docs/journal/`.

3.1.1 Entry point (the only one)

`resolve_combat(team, enemies, weather, *, node_id="") -> BattleResult` in `combat/resolve.py` is the **single public combat entry** (V.2). It is pure and deterministic: identical inputs -> byte-identical `BattleResult`. It delegates the setup to the shared `build_combat(...)` helper (the **one** wiring path) then runs the loop:

```
ctx, recorder = build_combat(team, enemies, weather, run_mods, node_id, seed) # resolve.py
# = compile_loadout (pieces, bus) -> assign_spawns -> recorder.register -> CombatContext
winner = run(ctx, recorder) # combat/engine.py
return recorder.build_result(winner) # combat/recorder.py
```

`build_combat` is reused (no parallel setup, T.37b) by:

- **resolve_boss_combat** (`combat/resolve.py`, T.12b/V.59 -- the **single src-side boss entry**; `tools/playtest/_common` delegates to it) -- same `build_combat`, then `attach_map_effect(map_effect_id, ctx, seed)` before run. Takes a `map_effect_id: str` (never a bosses/ content type => `combat/` stays content-import-free); byte-identical to the former `tools/` version (V.2). `CombatReplay/inspect_at_tick` accept the same `map_effect_id` to replay a boss fight; `CombatReplay.board_cells()` exposes the map-effect tiles as `(q,r,kind)` for the view overlay (V.1).
- **CombatReplay / inspect_at_tick** (`combat/replay.py`) -- `build_combat(..., with_recorder=False)`, then steps the `_step_combat` generator to read live state (see `Replay`).

3.1.2 The pieces

- **compile_loadout** (`loadout.py`) -- builds runtime `Pieces` from `Champion/Enemy` models, applies **Weather Favor** to `base_stats` (see [weather.md](#) -- the single application path), subscribes passive Hooks to a fresh `EventBus`, wires boss phase/death hooks. Returns `(pieces, bus)`.
- **Piece** (`piece.py`) -- the live combat entity (the *only* combat model; there is no separate snapshot type). Carries `base_stats`, modifiers, statuses, ability slots, position, meters, `crit_counter`. Effective stats via `piece.stat(name) = compute_stat` over `base + modifiers`.
- **CombatContext** (`combat/context.py`) -- the mutator API: the *only* way content touches the world (`deal_damage`, `apply_status`, `cast_ability`, ...). Holds the board state and `hex_distance`.
- **engine.run** (`combat/engine.py`) -- the tick loop (below). The **sole** tick engine (V.29); there is no second run.
- **BattleResultRecorder** (`combat/recorder.py`) -- subscribes to bus events and reconstructs `BattleResult` (outcome, survivors, damage, event stream, `piece_max_hp`, plus the `initial_pieces` board snapshot + board dims, T.37a). Observer-only: it never feeds combat math, so sims stay byte-identical (V.54).

3.1.3 Tick model

Time = **10 ms ticks** (`TICK_MS`); 1 round = 600 ticks (`ROUND_TICKS`, a presentation unit only). Each living piece accrues three meters per tick from its int stats: `action_energy += attack_speed`, `movement_energy += move_speed`, and mana via the **weighted-rank charge cycle** (T.29c, V.48): one slot is charged the full `mana_regen` per tick, cycle length `sum(slot.priority)`, each slot occupying `priority` positions (skip slots at `max_mana`) -> mana throughput = `mana_regen/tick` regardless of slot count. A meter fires at `ENERGY_THRESHOLD` (60 000) and **carries overflow** (it subtracts the threshold, not resets) so cadence is exact. Mana is per-slot: `mana_cost/max_mana` (default `2xcost/``start_mana/priority` are authored on the ability def (`ABILITY_MANA`), not a piece stat; a cast deducts `-= mana_cost` so overflow banks toward the next.

Multi-slot pieces (T.29d, V.49): a piece carries `active_abilities: list[str]` -- one `ActiveSlot` per id (single/null/multi are just list lengths; empty = a stat-stick with no mana bar). Roster ids are **discovered by convention** (`content.discover_abilities: {id}.active, {id}.active2, ...` sorted) unless a def sets `abilities=` explicitly (bosses' named kits, or `[]`). A multicaster's slots must differ in cost **or** priority (no lockstep simul-cast); the default is same cost + unique priorities (primary dominant), with high-tier **Ultimate** secondaries diverging by cost (2x) + priority proportional to cost.

Status durations are authored with `secs(x)` (seconds -> ticks, fractions OK, e.g. `secs(3.5)`) --

SECS (=100) for raw tick intervals. The stored value is always real ticks (no hidden runtime scaling; tick<->time stays honest). CC/DoT durations were re-tuned ~2x (1.5-3 s -> ~3-6 s) so effects don't expire between the slow ~5 s action cadence (60000/attack_speed).

- **Ordering** -- within a tick, triggered meters resolve in the canonical side-independent total order `_event_sort_key = (-round(attack_speedx1000), champion_id, load_order, kind)` (V.34, T.29-pre): faster attack-speed first (the quantized float key carries both whole + sub-integer speed in one term -- no separate `milli_AS`), then identity, then the seeded `load_order` (never team-then-enemy -> no side-A bias, B.14), then movement before action. Cadence reads `int(attack_speed)`. No RNG in the loop; `load_order` is a one-time seeded permutation (V.2/V.14). **Soft-CC**: slow stacks throttle action + movement meter *gain* by `_slow_factor = max(0.40, 1 - 0.15-stacks)` (V.53, B.25) -- not a gate, not the `stat`.
- **Movement** -- `_resolve_movement`: hold at threshold if in range or no enemies; else one BFS step toward the nearest in-range cell (`_next_step_toward`), carrying overflow; hold if no path. Gated by `BLOCKS_MOVEMENT` statuses.
- **Action** -- `_resolve_action`: cast an *unregistered* ability if mana full (fallback path), else auto-attack if an enemy is in range (`ctx.trigger_basic_attack`), else idle-hold. Registered abilities cast separately via `process_casts` -> `ctx.cast_ability` -- **at most one cast per window** (V.48 T4): among ready slots the highest priority casts (tie -> lowest slot index), the rest stay ready for later windows. Gated by `BLOCKS_ACTION/BLOCKS_ATTACK/BLOCKS_CAST`.
- **Per-tick upkeep** -- `_process_board_state` (slow tiles), `process_statuses` (DOT + decay on each status's cadence, then expiry), `expire_modifiers`, summon despawn.
- **Termination** -- ends when one side has no living piece. **Sudden death**: at `SUD-DEATH_TICK_START (= MAX_TICKS, 12 000)` the loop applies a stacking DOT (+3 stacks/engine-tick) that **damages once per second** (`dot_interval_ticks=100`, like every DOT -- V.25), so it escalates but still leaves room for a few last actions; `HARD_CAP_TICKS (= MAX_TICKS + 2 000)` is the absolute ceiling. **Outcome is survivor-based (V.60)**: WIN = team survivors & no enemy, LOSS = enemy survivors & no team, DRAW = **no survivors either side** (a true mutual wipe -- only via a simultaneous DOT/sudden-death pass). `timed_out` is an **independent flag**, not an outcome -- a sudden-death fight with a survivor is a real WIN/LOSS.

3.1.4 Damage pipeline

`_apply_hit / ctx.deal_damage` order:

```
raw = str_coeff-STR + int_coeff-INT
raw x= damage_modifier(attacker.affinity, target.affinity) # Affinity Clash, weather.md
if crit: raw x= CRIT_MULTIPLIER (1.5) # deterministic cadence
mitigated = raw x (1 - mit/(mit+100)) after penetration # _mitigated_damage
final = max(1, round(mitigated)) # true damage skips mitigation
```

- Coeffs (combat/engine.py): `auto = AUTO_STR_COEFF 1.0 / AUTO_INT_COEFF 0.2`; ability fallback = `ABILITY_STR_COEFF 0.2 / ABILITY_INT_COEFF 4.2`.
- **Crit is not random** -- `crit_counter` on the Piece increments per eligible hit and crits every `round(1/crit_chance)-th`, then resets. Shared autos/casts.
- **Mitigation** -- magical->resistance, physical->armor, true->unmitigated; `_effective_mitigation` applies `penetration_pct` then flat penetration, clamped `>= 0`. `damage_type` is a **closed vocabulary** {physical, magical, true} that `deal_damage` **validates** (raises on

anything else, V.58) -- a typo can't silently fall into the armor branch (B.29).

3.1.5 Result

`BattleResultRecorder.build_result` emits `BattleResult` with: `outcome` (WIN/LOSS/DRAW), `rounds/turns/duration_ticks`, `team_damage_dealt/_taken`, survivor id lists, `events` (full tick-ordered `BattleEvent` stream), `piece_max_hp` (`{id: int(max_hp)}` captured from the engine's pieces), and the combat-view layout (`initial_pieces`: per-piece `PieceSnapshot` identity + spawn-time position + mana profile, with `spawn_tick=0` for starters and the spawn tick for mid-combat summons; `board_width/board_height`).

Beat taxonomy (V.54, T.37a; ability T.37 follow-up). Every visible-state- changing beat emits exactly one `BattleEvent` from a single producer path: `move/attack/cast/ability/death` plus `heal/dot/status(applied)/ status_expire/spawn/despawn`. **cast vs ability are distinct:** `cast` is the *activation* marker (`amount=0`, "a piece casts", `_on_cast`); `ability` is the resulting *damage* (one per target hit, `_on_damage_dealt` when `tag == ABILITY`) -- first a `cast`, then per-target `ability` beats. Beats that actually change HP carry `hp_after/barrier_after` = the engine's post-event truth (read after `deal_damage` applies, V.28-correct: the amount is the full pre-barrier figure for DPS accounting): on `attack/ability/dot/heal`. The `ability/attack` amount is the **final post-mitigation** figure with `is_crit`

- `damage_type` (`physical/magical/true`, on `DamageEvent`) -- the single `ctx.deal_damage` chokepoint is the one producer (no separate ability handler). With `ability` added the stream is HP-complete; the view still reads bars from the live stepper (V.57) -- the beat's `amount/type` drives the floating *number*, not the bar. `turns` counts `attack+cast` only (`ability` excluded) => byte-identical. The **status (apply) beat fires once per acquisition** -- the recorder tracks a (`piece_id, status_id`) active set (cleared on `status_expire`) and skips re-applies/ refreshes (V.54), so sudden-death (re-applied every tick) + poison-restacks don't spam the stream (live stacks come from the stepper anyway, V.57). `expire_summon` fires `on_despawn` (distinct from `death`). The recorder is observer-only => sims byte-identical; only `combat_log` golden text re-baselines. `record_attack` (a dead parallel path) was removed -- `_on_attack_landed` is the sole attack producer.

Targeting footprints (T.12c, V.61) -- observer-only shape telemetry. Alongside beats, the recorder collects a `footprints: list[Footprint]` for the combat view's per-ability-shape VFX. The targeting helpers record the geometry they already compute: `enemies_in_radius/allies_in_radius/neighbors_of` -> a `circle` (center + radius), `line_targets` -> a `line` (origin + direction + length), via `ctx.note_footprint(kind, q, r, ...)` -> `on_footprint` -> `_on_footprint`. Capture fires **only while a cast is in flight** (`note_footprint` gates on `_current_cast_id`), so idle/AI/passive target queries don't record. It is **observer-only**: the helper returns its target list unchanged and, on the sim / `CombatReplay` path (no recorder subscribed), the bus fire no-ops => **byte-identical** (V.2/V.14, extends V.54). The `cast` + `ability` beats now carry the engine's `cast_id` so the view joins a footprint to its ability for element colour.

3.1.6 Rendering

`combat_log.py` turns a `BattleResult` into text purely from the result (it does **not** recompute anything): the HP trace prefers each event's `hp_after` (`barrier/DOT/heal-correct`, T.37a) and falls back to `piece_max_hp` + damage subtraction for legacy events. Used by the playtest CLIs and golden-snapshot tests.

3.1.7 Replay / forward stepper / inspect-at-tick (T.37b, T.37c)

Continuous combat state for a view is **recomputed, not recorded** (V.55). The engine loop is the **single_step_combat** generator (`combat/engine.py`, V.29), driven two ways -- no parallel loop:

- **run(ctx, recorder=None)** *drains* it to completion (the resolve path) -- byte-identical to the pre-T.37c monolithic loop (V.2/V.14).
- **CombatReplay(team, enemies, weather, *, run_mods=None)** *steps* it **forward** (`.step_to(tick)`, `.pieces()` -> `list[PieceView]`, `.tick`, `.winner`) for sequential playback -- drives one instance once, O(total ticks). Forward-only; `step_to` to an earlier tick raises.

The generator yields once per fully-processed tick (`yield 0` = after `on_combat_start`; `yield N` = after ticks 1..N), so a stepping consumer pauses mid-fight **before** the post-loop finalize (`end_combat/on_combat_end` never mutate the inspected state).

`inspect_at_tick(team, enemies, weather, *, run_mods=None, tick)` -> `list[PieceView]` is a random/backward-access wrapper (re-run from 0) **over** **CombatReplay** -- one driver, two shapes. Each reads hp, barriers, per-slot mana, **effective stats** via `piece.stat()` (STR/AS ramp included), statuses, position into frozen read-only `PieceView/SlotView/StatusView` structs. `run_mods` is **cloned** (deep-copy of mutable `augment_state`) so replay never mutates the caller. Raw `Piece/Flet` never escape `src/game/` (V.1). No per-tick state is stored in `BattleResult` (avoids T.14 save bloat + stat-drift).

This live state is the combat view's resource-truth source (V.56/V.57), NOT the recorded event stream -- even though the stream is now HP-complete (the `ability` beat closed the B.28 gap), the view keeps **one** source of truth (the stepper) so bars can't drift from a dual pipeline; the stream's `amount/type` drives the floating *number*, the stepper drives the *bar*. The stream is **animation cues + action-queue projection** only. `move/` `spawn` beats carry structured `dest_q/dest_r` int coords (T.37c), not a parsed note string.

3.1.8 Invariants this system owns

- **V.2** -- combat is a pure, seeded function; the ability/passive/status framework is invoked *through* `resolve_combat`, never alongside it.
- **V.29** -- exactly one tick loop (`combat/engine.py`); no parallel engine.
- **V.14** -- determinism: every "chance" mechanic uses a cadence counter (`crit_counter`), never RNG.
- **V.54** -- event-stream completeness: every visible-state-changing beat emits exactly one `BattleEvent` (T.37a).
- **V.55** -- view state is recomputed by replay, never recorded as per-tick keyframes; the forward `CombatReplay` stepper + `inspect_at_tick` are pure + clone `run_mods` (T.37b, forward stepper T.37c).
- **V.56/V.57** -- the combat view's resource truth (hp/mana/stats/position) is the live replay, **never** the event stream's partial `hp_after` fields (incomplete for ability burst, B.28); the stream is animation cues + queue projection (T.12).
- **V.58** -- `damage_type` is a closed vocabulary `{physical, magical, true}`; `deal_damage` validates + raises on anything else so a typo can't be mis-mitigated as physical (B.29).
- **V.61** -- targeting footprints are observer-only telemetry for the combat view: helpers record geometry via `ctx.note_footprint` -> `on_footprint` only while a cast is in flight; the recorder stamps footprint records on `BattleResult`; capture never changes targeting results or damage

=> byte-identical (T.12c).

3.1.9 File map

Concern	File
Public entry + shared <code>build_combat</code> wiring	<code>combat/resolve.py</code>
Single tick loop (<code>_step_combat</code> generator) + run drain, pathing, damage, spawns, constants	<code>combat/engine.py</code>
Forward stepper <code>CombatReplay</code> + <code>inspect_at_tick</code> (recompute state at a tick)	<code>combat/replay.py</code>
Mutator API (the only way to touch the world)	<code>combat/context.py</code>
Event -> <code>BattleResult</code> (incl. footprint telemetry)	<code>combat/recorder.py</code>
Targeting helpers + footprint capture	<code>targeting.py</code> (<code>ctx.note_footprint</code> in <code>combat/context.py</code>)
Model -> <code>Piece</code> compile + weather + passives	<code>loadout.py</code>
Runtime piece	<code>piece.py</code>
Boss wiring (map effect)	<code>tools/playtest/_common.py::resolve_boss_combat</code>

3.2 Effects -- hooks, modifiers, statuses, registries

Status: `LIVING` -- must match `src/game/effects.py`, `events.py`, `status.py`, `registries.py`, `piece.py`, and `combat/context.py`. Audited by `/check`. **Scope:** the T.20 effect substrate -- how content (abilities/passives/traits) changes combat without touching the loop. **Reconciled:** 2026-06-05.

Citations by symbol, not line. Design rationale (frozen): `docs/design/systems/effect_systems_design.md`, `passive_system_proposal.md`. Catalog: see [abilities.md](#), [traits.md](#).

3.2.1 The shape

Content never calls the tick loop. It registers **hooks** on an `EventBus`; the loop fires events; hooks mutate the world **only** through the `CombatContext` API. Three data primitives carry the change:

- **Modifier** (`effects.py`, frozen) -- a stat delta (`stat`, `op`, `value`), `op` in `{add, mul, set}`, applied by `compute_stat` as $(base + \Sigma add) \cdot \Pi mul$, `set` overriding. Carries a `Lifetime` (`PERMANENT/COMBAT/TIMED`); `TIMED` uses `expires_at_tick`. Lives in `Piece.modifiers`; read via `piece.stat()`.
- **StatusInstance** (`status.py`) -- a stacking, expiring effect on `Piece.statuses`, defined by a `StatusDef` in `STATUS_DEFS` (`DOT` cadence, decay, `StackBehaviour`, gates). Tick upkeep is `engine.process_statuses`.
- **EffectBundle** (`effects.py`) -- a named group of hooks + modifiers a passive/item installs at once; `ctx.register_bundle(owner, bundle)`.

3.2.2 EventBus

`EventBus` (`effects.py`): `subscribe(Hook)` -> `hook_id`, `fire(name, event, *, ctx)`, and `fire_reducing(...)` for value-mutating chains (e.g. damage pre-mods). A Hook has an event

name, handler, priority (higher first; the recorder subscribes at -1000 to record last), and a HookScope (PER_HIT / ONCE_PER_CAST / ONCE_PER_TARGET / ONCE_PER_COMBAT).

Events fired by the engine/context (payloads in events.py): on_combat_start, on_tick, on_attack_start, on_attack_landed, on_damage_pre, on_damage_dealt, on_damage_taken, on_ability_damage, on_cast, on_cast_complete, on_heal, on_status_applied, on_status_expired, on_kill, on_death, on_spawn, on_combat_end.

3.2.3 StatusGate -- how statuses disable actions

StatusGate (status.py) values, checked by piece.is_gated(gate) in the loop:

Gate	Blocks
BLOCKS_ACTION	all meter advancement (stun, frozen)
BLOCKS_CAST	ability casts (silence)
BLOCKS_ATTACK	auto-attacks (disarm)
BLOCKS_MOVEMENT	hex movement (root, frozen)

3.2.4 CombatContext -- the mutator API

The **only** way content touches the world (combat/context.py).

- Damage / heal / shield: deal_damage(source, target, amount, SourceTag, *, crit=None, damage_type=...), heal, grant_barrier.
- Statuses / mods: apply_status, remove_status, apply_modifier, register_bundle.
- Actions: trigger_basic_attack(attacker, target, mult=1.0), cast_ability(actor, slot_idx=0), gain_mana/spend_mana, teleport, spawn, expire_summon, kill, end_combat.
- Queries: enemies_of/allies_of, all_pieces/living_pieces, both_sides_alive, current_tick, rng (seeded -- for non-combat-affecting choices only).

SourceTag (basic_attack / ability / dot / status / reflect / true / item_proc) tags every damage instance so hooks can filter what they react to.

3.2.5 Registries -- id -> handler

registries.py holds ABILITY_REGISTRY, PASSIVE_REGISTRY, ITEM_REGISTRY, TRAIT_REGISTRY, AUGMENT_REGISTRY. Content registers via @register_active, @register_passive, @register_item, @register_trait (and register_active_simple for declarative single-target spec abilities). Roster ability ids are prefixed (champ_x.active); every id must resolve (CI-guarded). compile_loadout imports the abilities package to trigger the decorators.

3.2.6 Invariants this system owns

- **V.14 / determinism** -- "chance" effects use a cadence counter (crit_counter, Status-Def.dot_interval_ticks), never RNG.
- **Boundary** -- combat/ never imports content at module scope; content reaches combat only through CombatContext + the registries.

3.2.7 File map

Concern	Symbol
Bus, Hook, HookScope, Modifier, EffectBundle, compute_stat, Lifetime, SourceTag	effects.py
Typed event payloads	events.py (DamageEvent, AttackEvent, DeathEvent, ...)
Status defs/instances, gates, STATUS_DEFS, StackBehaviour	status.py
Mutator API	combat/context.py (CombatContext)
Per-piece modifier/status/barrier state, crit_counter	piece.py
id -> handler registries + @register_*	registries.py
Status tick upkeep	combat/engine.py::process_statuses / expire_modifiers

3.3 Weather -- Favor & Affinity Clash

Status: **LIVING** -- must match src/game/weather_effects.py + loadout._apply_weather_to_piece. Audited by /check. **Scope:** the two decoupled weather systems and the single place each is applied. **Reconciled:** 2026-06-05 @ refactor/combat-engine-single-source.

Six weather states (V.5): CLEAR, CLOUDY, MIST, RAIN, SNOW, THUNDER. Each piece carries exactly one **affinity: WeatherState** (V.6). CLEAR sits outside the predator/prey ring and is inert in both systems. The two systems are **never summed**:

3.3.1 1. Weather Favor -- combat_modifier

"Does the node weather suit my affinity?" A 5-tier stat buff/debuff (combat_modifier(affinity, weather) -> CombatModifier) driven by the directional predator/prey ring. Applied **once at combat init**, in exactly one place: loadout._apply_weather_to_piece. As of **T.29-pre (V.42)** it no longer folds into base_stats -- it emits source="weather:<state>" **Modifiers** (*_mult!=1.0 -> ("<stat>", "mul", mult), attack_range_delta -> ("attack_range", "add", delta)) applied via apply_bundle, so weather composes through compute_stat (base+Σadds)xΠmults like every other source -- uniformly attributable (stat_breakdown, V.45) and it scales item/augment adds. Values are now **unrounded floats** (not round(valuexmult)); attack_range underflow is held

= 1 by the _STAT_FLOORS clamp in compute_stat (V.43); HP is reconciled (max_hp = hp = stat("hp")) afterwards since resources are never Modifier'd directly. There is no other application path -- the old weather_effects.apply_weather snapshot and the CombatPieceState model it built were removed (one source of truth).

3.3.2 2. Affinity Clash -- damage_modifier

"Do I beat this enemy?" A per-hit multiplier damage_modifier(attacker_affinity, defender_affinity) applied on **every damage instance** inside the damage pipeline (see [combat.md](#)). It depends on the defender, so it can't be pre-snapshotted -- it's resolved per hit, not at init.

3.3.3 Why decoupled

Favor asks about the *node*; Clash asks about the *opponent*. Keeping them separate means weather tuning and matchup tuning don't entangle. Both read the same predator/prey ring (`ring_relation`) but apply at different times to different magnitudes (`combat_modifier` stat packs vs `damage_modifier` multiplier).

3.3.4 Also here

`shop_weight(affinity, weather)` -- prep-shop pull weight by ring relation (content economy, not combat).

3.3.5 File map

Concern	Symbol
Ring relation (predator/prey)	<code>weather_effects.ring_relation</code>
Weather Favor stat pack	<code>weather_effects.combat_modifier</code> -> <code>CombatModifier</code>
Weather Favor application (only path)	<code>loadout._apply_weather_to_piece</code>
Affinity Clash per-hit multiplier	<code>weather_effects.damage_modifier</code>
Shop pull weight	<code>weather_effects.shop_weight</code>

3.4 Formation -- enemy spawn placement

Status: **LIVING** -- must match `src/game/formation.py` (called by `combat/engine.py::assign_spawns`). Audited by `/check`. **Scope:** how a generated enemy squad is placed on the board. **Reconciled:** 2026-06-05 @ refactor/combat-engine-single-source.

Deterministic, role-aware placement of the enemy squad on the right of the board (columns 7-9). Pure function -- no RNG, no I/O (V.1, V.2). Player champions are placed by the engine on the left (index-packed); this module handles **enemies only**.

3.4.1 Entry

`plan_enemy_formation(pieces, enemy_defs_by_id, *, boss_position=None, board_height=BOARD_HEIGHT)` -> `dict[int, tuple[int, int]]` -- maps each enemy's `formation_index` -> (col, row). Called by `combat/engine.py::assign_spawns`, which builds a lightweight `_FormationEnemy` shim per enemy (not a full `Piece`), calls this, then writes `formation[enemy.formation_index]` back onto `piece.position_q/position_r`.

3.4.2 The input contract -- `FormationPiece`

The planner reads exactly three attributes, declared as a structural Protocol `FormationPiece` in `formation.py`:

- `piece_id`: str -- identity (sorted by, for determinism).
- `tier`: int -- boss/role weighting (tier 10 = boss).

- `formation_index`: int -- the board-slot key the returned `placements` dict is keyed by. **Not** the combat tiebreak: T.33a split the old `speed_tiebreaker` into `formation_index` (this placement key) and `load_order` (the `_event_sort_key` tiebreak, see [combat.md](#)). Formation only touches `formation_index`.

`combat/engine.py::_FormationEnemy` and the tests satisfy this structurally -- formation imports **no** piece model, keeping it pure (V.1).

3.4.3 Placement policy

`classify_role(enemy_def: EnemyDef) -> PlacementRole` buckets each enemy by `enemy_def.durability` and `enemy_def.reach`:

PlacementRole	Column	Rule (durability / reach)
FRONTLINE	COL_FRONT (7)	<code>durability in ("tanky_hp", "tanky_arm")</code>
FLANK	COL_MID/COL_BACK (8-9) edge rows	<code>reach == "melee" and durability == "squishy"</code>
MIDLINE	COL_MID (8)	<code>reach == "melee" (else)</code>
BACKLINE	COL_BACK (9)	<code>ranged (fallthrough)</code>

Rows fill center-out (`_center_out_rows`); overflow spills to adjacent columns; `_nearest_free` is the last-resort packer. Bosses (tier 10) take the per-boss authored `boss_position` via `_place_boss`.

3.4.4 File map

Concern	Symbol
Plan a squad	<code>formation.plan_enemy_formation</code>
Role classification	<code>formation.classify_role -> formation.PlacementRole</code>
Input contract	<code>formation.FormationPiece</code> (Protocol: <code>piece_id/tier/formation_index</code>)
Columns	<code>formation.COL_FRONT/COL_MID/COL_BACK</code>
Row packing	<code>formation._center_out_rows</code> , <code>formation._nearest_free</code>
Boss slot	<code>formation._place_boss</code>
Caller (builds <code>_FormationEnemy</code> , writes coords)	<code>combat/engine.py::assign_spawns</code>

3.5 Encounter & board -- squad generation, bosses, map effects

Status: **LIVING** -- must match `src/game/encounter.py`, `bosses/data.py`, `map_effects.py`, `board.py`. Audited by `/check`. **Scope:** seed-deterministic enemy/boss squad generation, difficulty, and the board-cell state combat reads. **Reconciled:** 2026-06-05.

Citations by symbol, not line. Placement of the generated squad is [formation.md](#). Design rationale (frozen): `docs/design/tasks/t19_*`, `t21_*`.

3.5.1 Determinism -- every roll derives from the seed

All procedural choices come from `derive_seed(run_seed, node_index, channel)` (V.14) -- same `(run_seed, node_index, channel) -> identical squad`. Per-domain seed helpers: `augment_seed`, `supply_seed`, `shop_seed`, `economy_seed`. The RNG is Python Random seeded from that derived value; no global randomness.

3.5.2 Difficulty

`next_dc(current_dc) -> float` advances the difficulty class along the run; `dc_name(dc)` labels it. DC feeds tier/level weighting in squad rolling.

3.5.3 Squad generation

- `roll_squad(...)` -- the core roller: weighted tier pick (`_tier_weight` by stage), level pick (`_pick_level`), affinity slotting (`_affinity_slots` by stage affinity), role-composition guard (`_check_composition` over `_is_tanky/_is_support/_is_dps`), via `_weighted_pick` over a `filter_pool` of `EnemyDefs`. Builds `Enemy` instances with `_instantiate_enemy`.
- `generate_fight(...)` -- a standard fight encounter.
- `generate_reward(...)` -- reward-node contents (enemy squad / supplies).
- `generate_reward_loot(run_seed, node_index) -> RewardLoot` -- seed-deterministic item drop for REWARD nodes; uses channel `CH_REWARD = 8`. 60% one component, 25% one core item, 15% two components. Added in T.29a. See [items.md](#).
- `generate_challenge(...)` -- challenge nodes; can pull champion-derived enemies (`_champion_def_to_enemy`) and yields a `ChallengeReward`.
- `generate_boss_encounter(run_seed, node_index, stage) -> BossEncounterResult` -- the boss squad + its map effect.
- `generate_node_reward(run_seed, node) -> NodeReward | None` (T.38, V.70) -- the **single reward-payload source**, type-dispatched (mirrors `node_encounter`): **REWARD** -> `generate_reward_loot` items; **CHALLENGE** -> the `generate_challenge` reward (amber 2 x `stage_index`, both components, `tempest_bonus`, `champion_offer`); **all other types** -> `None`. Uses `node.weather` (the node's `default_weather`, not live API weather) so the payload is byte-identical to the reward `node_encounter` *discards* (the fight-build path stays squad-only; the resolve path `economy.apply_node_result` owns the reward). `NodeReward` fields -> `Run.inventory / amber / tempest / pending recruit`. Applied **on win only** => a loss is structurally reward-zeroed.

3.5.4 Bosses

`bosses/data.py`: `BossDef` (`kit`, `BossCastEntry` list, `map_effect_id`, `spawn_position`), the `BOSS_DEFS` registry, and `BossEncounterResult` (`.all_enemies` property = boss + adds, `.map_effect_id`). Combat wiring for a boss fight (attach the map effect before the loop) is `tools/playtest/_common.py::resolve_boss_combat` -- see [combat.md](#).

3.5.5 Board & map effects

`board.py` `BoardState` carries per-cell state the loop reads each tick: `slow_cells` (+ `is_slow(q, r)`, consumed by `engine._process_board_state`), `fog_range` (+ `is_in_fog_range(...)`, read by `targeting.py`), and `CellModifier`. `map_effects.py` defines `MapEffect` (base; `register(bus)` wires it) and the concrete

effects -- SunlitTilesEffect, FogEffect, HazardTilesEffect, DefensiveLeyEffect, FloodLanesEffect, SlowTilesEffect -- keyed in MAP_EFFECT_CLASSES; build_map_effect(effect_id, board, seed) constructs one. loadout.attach_map_effect (see [combat.md](#)) builds

- registers it onto a CombatContext.

3.5.6 File map

Concern	Symbol
Seed derivation	encounter.derive_seed (+ augment_seed/supply_seed/shop_seed/economy_seed)
Difficulty	encounter.next_dc / dc_name
Squad roll	encounter.roll_squad, generate_fight/generate_reward/generate_challenge/generate_boss_encounter
Boss data	bosses/data.py (BossDef, BOSS_DEFS, BossEncounterResult)
Board state	board.py (BoardState, CellModifier)
Map effects	map_effects.py (MapEffect, MAP_EFFECT_CLASSES, build_map_effect)

3.6 Stat scaling

Status: **LIVING** -- must match src/game/scaling.py + content.py stat curves. Audited by /check. **Scope:** the power curve P(tier,level), the three stat-scaling classes, baseline parity, and the combat tie-break. **Reconciled:** 2026-06-05 (T.33a/b).

3.6.1 Power curve

- power(tier, level) = 2 ** ((tier-1)/3 + triplings[level]), triplings = {1:0, 2:1, 3:3} (T.18).
- stat_multiplier(tier, level, exponent=PRIMARY_EXPONENT) = power(tier, level) ** exponent.

3.6.2 Three scaling classes (T.33, V.34)

Every base stat is in exactly one class; both curves ride the same power curve at a different exponent:

class	stats	exponent	per-tier	T1L1->T10L3
PRIMARY_SCALABLE_STATS	hp strength intelligence armor resistance	PRIMARY_EXPONENT=0.5 (sqrt(power))	~= x1.122	x8
SECONDARY_SCALABLE_STATS	attack_speed move_speed mana_regen threat	SECONDARY_EXPONENT=0.0857	~= x1.02	x1.428
FLAT_STATS	attack_range	--	--	--

crit_chance/penetration/penetration_pct are ratios, off the scaling model. SCALABLE_STATS is a deprecated alias of the primary tuple. level_scale_stats(stats, tier, level) applies both curves in place -- the single source of truth for the four builders (content._build_champion/_build_enemy, encounter._instantiate_enemy/_champion_def_to_enemy). threat/move_speed stay **off** the HP-DPS power budget (V.33, B.6).

3.6.3 Quantities: int except attack_speed

Stored quantities (hp, damage, mana, move_speed/mana_regen/threat, costs, energy meters) are **int**; the ratio stats **and** **attack_speed** are float. **attack_speed** is a float (T.29-pre, amends V.34): cadence reads `int(attack_speed)`, and the sub-integer precision for same-tick ordering derives from the **same** value via `round(attack_speed x 1000)` -- there is no separate `milli_AS` field. Because weather now applies as mul modifiers (not a base fold), **attack_speed** stays float through level **and** combat.

3.6.4 Speed-stat baseline parity (#39, V.35)

`_BASE_STATS attack_speed == move_speed == mana_regen == 100` so the three speed stats compare directly as power investments. The per-meter capacitor is deliberately unequal and non-player-facing: `mana mana_cost baseline 300_000 » action/move ENERGY_THRESHOLD = 60_000` (a cast $\sim$ 5 autos). `mana_cost 300_000` (vs cadence-neutral 360_000) is a deliberate $\sim$ 20% mage buff; `move_speed 90->100` a $\sim$ 11% movement buff. **T.29c (V.48)**: cast cost is no longer a per-piece `ability_cost` FLAT stat -- it is authored **per-ability** on the ability def (`ABILITY_MANA`, default 300_000); bosses register their own (380k-520k). `max_mana` defaults to `2xmana_cost` (overload headroom).

3.6.5 Speed axis -- 7 levels (T.33b)

`_SPEED` (content.py): `leaden / heavy / steady / hybrid / brisk / speedy / blinding`, slow->fast. Faster $\Rightarrow$ $\uparrow$ move_speed, $\downarrow$ primary_stat (softer per-hit/cast), and a tempo gain that **routes by playstyle**: auto/hybrid pieces get the full **attack_speed** mult; **ability** casters get **half** the AS deviation applied to *both* **attack_speed** and **mana_regen** (more, softer casts) -- speed no longer touches resistance. Roughly power-neutral (cadence up, per-hit down). **hybrid** is the neutral centre (omitted from `role_code`). Widening 3->7 took the role-code space to 1512 combos (V.32); `classify_role` ignores speed, so role *titles* are unaffected. The full 60-champion + 60-enemy roster is assigned across the 7 levels by theme + within-tier AS spread (54/60 champions distinct-AS within their tier).

3.6.6 Combat tie-break (V.34, fixes B.14)

Same-tick action order is the canonical side-independent total order in `_event_sort_key` (combat/engine.py): `(-round(attack_speed x 1000), champion_id, load_order, kind)` -- the quantized AS key (T.29-pre) is monotonic in the float **attack_speed**, so it subsumes the old coarse `-AS_int` level and the separate `milli_AS` field in one term.

- `load_order` -- a seeded, side-independent permutation assigned in `compile_loadout` (never team-block-then-enemy), so equal-AS ties never systematically favour the player team (the old B.14 bug). Renamed from the overloaded `speed_tiebreaker`; the formation-position key is now `formation_index`.

3.6.7 Where it lives

- `scaling.py` -- power curve, the three class tuples + exponents, `stat_multiplier`, `level_scale_stats`.
- `content.py` -- base stat blocks, axis multipliers, `compose_stats` (**attack_speed** kept float), `_ABILITY_COST`.
- `combat/engine.py` -- `_event_sort_key`. `loadout.py` -- `load_order/formation_index` assignment, weather application.
- `tools/gen_role_matrix.py` -- regenerates `docs/design/tasks/t32_role_matrix.txt`.

3.7 Weather API -- fetch, cache, refresher

Status: **LIVING** -- must match `src/api/weather.py`, `api/cache.py`, `api/refresher.py`. Audited by `/check`. **Scope:** OpenWeather client, per-city cache states, and the 3-stream tick refresher (≤ 3 calls/min). **Reconciled:** 2026-06-05.

Citations by symbol, not line. Design (frozen): `docs/design/tasks/t6_*`, `t7_*`. This layer is the *only* I/O in the project -- `src/game/` never touches it (V.1).

3.7.1 Client -- `api/weather.py`

`WeatherClient(api_key=None)` reads `OPENWEATHER_API_KEY` from env (never logged, V.3). `fetch_weather(...)` hits OpenWeather by lat/lon and returns a frozen `WeatherResult` (the mapped `WeatherState` + metadata). On any error it returns `_fallback_result(fallback, error)` -- a `WeatherResult` flagged `is_fallback` (the city-default weather) rather than raising. The OpenWeather id -> `WeatherState` mapping lives on `WeatherState.from_openweather_id(models.py)`.

3.7.2 Cache -- `api/cache.py`

`WeatherCache` holds one `CacheEntry` per `city_id` with a `CacheState`:

CacheState	Meaning	fetched_at
UNKNOWN	never fetched	None
LIVE	real fetch succeeded	set
SUBSTITUTE	fetch failed -> city-default weather	set

`set_live` / `set_substitute` update an entry; `fetch_and_cache(client, cache, city_id, ...)` does the round trip -- success -> `set_live`, `is_fallback` -> `set_substitute`. Advancing to an `UNKNOWN` city triggers a synchronous fetch.

3.7.3 Refresher -- `api/refresher.py`

`WeatherRefresher` ticks once per `tick_interval` (default 60 s) on a daemon thread (HTTP off the main thread, V.4). Each `tick()` picks ≤ 3 **unique** cities across three streams and fetches each, returning the fetched `city_ids`:

- **A** -- full round-robin over all cities.
- **B** -- round-robin within the window `[current+1 .. current+6]` (`_b_pointer`).
- **C** -- uniform random (seedable via `rng_seed` for tests).

Dedupe across streams keeps it to ≤ 3 API calls/min.

3.7.4 Node weather lifecycle -- persisted on the Run (T.39, V.73)

The cache/refresher are **fetch-scheduling + freshness** only; the **persisted source of truth is the Run Node**. Each Node (`game/models.py`) carries:

Field	Meaning
<code>weather: WeatherState</code>	the effective game weather all systems read -- <code>default_weather</code> placeholder until a live fetch overwrites it (T4 §2)
<code>weather_state: NodeWeatherState</code>	UNKNOWN (never fetched -> display ?) / LIVE / SUBSTITUTE -- distinct from <code>api.cache.CacheState</code> (game-local enum; importing <code>CacheState</code> would cycle)
<code>weather_locked: bool</code>	frozen for the run; the refresher skips it

The Trail copies fetched cache values onto the Run via the **pure game-side mutators** `Run.set_node_live_weather(node_index, weather, *, is_substitute)` and `Run.lock_node_weather(node_index)` only -- the cache/refresher never touch game state (V.10). `set_node_live_weather` is a **no-op on a locked node**. The **current** node's weather **locks at the Trail->Prep transition** (`trail.py::play_next` -> `lock_node_weather`, persisted by `main._push_prep`'s `save_run`); an UNKNOWN node freezes `default_weather` flagged SUBSTITUTE. Locking before the fight keeps the CHALLENGE squad roll + `generate_node_reward` byte-identical (load-bearing for V.70). Pre-T.39 saves lack the two new fields -> UNKNOWN/False on read (no schema_version bump).

3.7.5 Invariants this system owns

- **V.3** -- the API key is never logged.
- **V.4** -- all HTTP runs on a worker thread; a fetch failure never crashes (falls back to substitute weather).
- **V.1** -- game logic has zero dependence on this layer; it consumes only the resulting `WeatherState`.

3.7.6 File map

Concern	Symbol
HTTP client + fallback	<code>api/weather.py</code> (<code>WeatherClient.fetch_weather</code> , <code>WeatherResult</code> , <code>_fallback_result</code>)
Per-city cache	<code>api/cache.py</code> (<code>WeatherCache</code> , <code>CacheEntry</code> , <code>CacheState</code> , <code>fetch_and_cache</code>)
3-stream tick loop	<code>api/refresher.py</code> (<code>WeatherRefresher.tick</code>)
id -> state mapping	<code>models.py::WeatherState.from_openweather_id</code>
persisted node lifecycle	<code>models.py</code> (<code>Node.weather/weather_state/weather_locked</code> , <code>NodeWeatherState</code> , <code>Run.set_node_live_weather/lock_node_weather</code>); write-through + lock in <code>ui/views/trail.py</code>

3.8 Save / serialization

Status: **LIVING** -- must match `Run/BattleResult` (de)serialization in `src/game/models.py` + the file-I/O layer in `src/game/save.py`. Audited by /check.

Scope: how game state round-trips to/from JSON, the back-compat rules, and the file read/write contract. **Reconciled:** 2026-06-05.

Citations by symbol, not line. The (de)serialization contract lives on the **model dataclasses**; the **file I/O** (atomic write, schema gate, typed errors) lives in `game/save.py` (T.14).

3.8.1 The model is the schema

Every persisted dataclass has `to_dict()` -> dict and `@classmethod from_dict(payload)`: Run, Champion, Enemy, Node, BattleEvent, BattleResult. Run is the root -- it holds all game state and serializes its nested objects.

Run fields (root): `run_id`, `schema_version` (≥ 1 , validated), `seed`, the route Nodes, the champion roster, `amber` (economy), `battle_log` (list[BattleResult]), `status`. `Run.to_dict` nests each child's `to_dict`; `from_dict` rebuilds them.

3.8.2 Back-compat rules (the part that bites)

`from_dict` must tolerate older payloads. Current rules:

- **amber** <- **gold** -- old saves used "gold"; `Run.from_dict` reads the legacy `gold` key when `amber` is absent (B.4).
- **BattleResult.piece_max_hp** -- added later; `from_dict` defaults it to {} for pre-field saves (the combat-log HP trace is just empty for them). See [combat.md](#).
- New optional fields follow the same pattern: `payload.get(key, default)`, never a hard `payload[key]`, so old saves still load.

`schema_version` exists to gate breaking migrations; bump it when a change can't be handled by a `.get` default.

3.8.3 File I/O (game/save.py, T.14)

The disk layer over the model contract. Imports only `json/os/pathlib` -- no Flet (V.1).

- **CURRENT_SCHEMA_VERSION** (= 1) -- the single source for the version this build writes/reads (was hardcoded per-Run before T.14).
- **save_run(run, path)** -- `run.to_dict()` -> JSON, **atomic**: writes `<path>.tmp`, `flush+fsync`, then `os.replace` onto `path`; auto-creates parent dirs; removes the temp on any failure. Readers never see a partial file.
- **load_run(path)** -- reads + gates on `schema_version` (see errors below), then `Run.from_dict`. The migration hook sits between the gate and `from_dict`.
- **default_save_dir()** -- platform app-data dir (`%APPDATA%` / macOS Application Support / `$XDG_DATA_HOME`) .../tempest-fauna-trail/saves. Helper only; core fns take an explicit path. Does **not** create the dir (save does).
- **Errors:** `SaveError` (base) -> `CorruptSaveError` (bad JSON, missing/mistyped required keys incl. `schema_version`, or a payload that fails Run validation -- `from_dict`'s `ValueError/KeyError/TypeError` are wrapped), `UnsupportedSchemaError` (`schema_version > CURRENT_SCHEMA_VERSION`). `FileNotFoundError` is **not** wrapped -- callers branch on "no save yet".

3.8.4 Invariant

- Round-trip identity: `from_dict(x.to_dict()) == x` for current data, and old-shaped payloads load without error (back-compat). /check, `tests/game/test_models.py`, and `tests/game/test_save.py` (through disk) guard this.
- File safety: `save_run` is atomic (temp + `os.replace`); `load_run` never returns a `Run` from a future-schema or invalid file -- it raises a typed `SaveError`. (V.36)

3.8.5 File map

Concern	Symbol
Root state + nesting	<code>models.py::Run.to_dict</code> / <code>Run.from_dict</code>
Battle records	<code>models.py::BattleResult.to_dict</code> / <code>from_dict</code> (piece_max_hp optional)
Per-entity	<code>Champion</code> / <code>Enemy</code> / <code>Node</code> / <code>BattleEvent</code> <code>.to_dict</code> / <code>.from_dict</code>
Legacy key read	<code>Run.from_dict</code> gold->amber (B.4)
File read/write	<code>save.py::save_run</code> / <code>load_run</code> (atomic, schema-gated)
Save version constant	<code>save.py::CURRENT_SCHEMA_VERSION</code>
Default save location	<code>save.py::default_save_dir</code>
Typed errors	<code>save.py::SaveError</code> / <code>CorruptSaveError</code> / <code>UnsupportedSchemaError</code>

3.9 Items -- engine, components, combined, equip, reward drops

Status: **LIVING** -- must match `src/game/items/`, `src/game/loadout.py`, `src/game/models.py`, `src/game/encounter.py`, `src/game/registries.py`. Audited by /check. **Last reconciled:** **2026-06-16** (T.29a-d complete -- components, combined, emblems, special run-actions, mana primitive, multi-slot).

Design rationale (frozen): `docs/design/tasks/t29_item_engine_plan.md`. Content roster (frozen): `docs/design/content/item_catalog.md`.

3.9.1 Package layout

```
src/game/items/
__init__.py    -- re-exports BASE_COMPONENTS, SPIRIT_GEM, RECIPE_MAP, combine;
                importing this module triggers all @register_item side-effects
base.py        -- BASE_COMPONENTS: frozenset[str] (8 component IDs), SPIRIT_GEM
recipes.py     -- RECIPE_MAP: dict[frozenset[str], str] (36 entries), combine()
combined.py    -- @register_item factories: 8 raw components + 16 core + 20
                combined (T.29b) = 44 in ITEM_REGISTRY
emblems.py    -- 6 Kinship emblems (T.29b; ITEM_REGISTRY total = 50)
special.py     -- 5 RUN_ACTION_REGISTRY special items (T.29b, operate on Run)
```

ITEM_REGISTRY (in `src/game/registries.py`) maps `item_id` -> `Callable[[Piece], EffectBundle]`.

3.9.2 Data structures

3.9.2.1 Champion.items (models.py)

```
@dataclass
class Champion:
```

```
items: list[str] = field(default_factory=list) # <=3 item IDs, validated
```

Invariants (V.23):

- Length ≤ 3 (validated in `Champion.__post_init__`).
- Every string is non-empty.
- Items persist across combats on the `Champion`; each combat rebuilds from scratch.

3.9.2.2 RECIPE_MAP (recipes.py)

A `dict[frozenset[str], str]` mapping a pair (or singleton, for same-component recipes) of component IDs to the combined item ID.

- 36 entries total (8x8 upper triangle + diagonal).
- Same-component recipes use a **single-element frozenset** because `frozenset({"fang", "fang"}) == frozenset({"fang"})`.
- `spirit_gem` is **not** in `BASE_COMPONENTS`; it is a separate special-item. `combine("spirit_gem", c)` returns `c`'s emblem (`base.KINSHIP_OF`) -- 6 mapped components craft emblems; `ward-pelt/keen_claw` -> `None` (T.29b).

```
def combine(a: str, b: str) -> str | None:
```

Returns the combined item ID or `None` if no recipe exists.

3.9.3 Prep equip seam (game/inventory.py, T.23b)

The Prep view moves items between `Run.inventory` and `Champion.items` **only** through `game/inventory.py` (V.63 -- never inline):

- `equip_item(run, champion, item_id) -> bool` -- consumes one from inventory onto the champion. **Auto-combines on double-equip:** if the champion already holds a component that pairs with `item_id` into a recipe (`items.combine`), the two fuse into the combined item in a single slot (works even at the 3-item cap); otherwise the item takes a free slot (≤ 3 , the `Champion` cap). The combine partner is the **first** held item that pairs -- deterministic, no RNG (V.2).
- `unequip_item(run, champion, item_id) -> bool` -- returns the item whole to inventory (no de-combine).

3.9.4 Equip pipeline (combat-side)

`compile_loadout` (loadout.py) applies items at **step 2.5** -- after weather modifiers (step 2) but before trait resolution (step 3).

```
step 1: compose_stats
step 2: weather modifiers
step 2.5: item bundles      <- NEW (T.29a)
step 3: trait resolution
...
```

Flow:

1. `piece_from_champion` copies `champion.items` -> `piece.items` (`Piece.items` is a `list[str]`; defaults to `[]`).

2. At step 2.5, `compile_loadout` imports `src.game.items` (triggers registry population) then iterates `piece.items`; for each item ID calls `ITEM_REGISTRY[item_id](piece) -> EffectBundle`, then `apply_bundle(piece, bundle)`.
3. Modifiers from item bundles use `Lifetime.COMBAT` (pieces are rebuilt each combat -- no cross-combat accumulation).

Enemies carry **no items**; `piece_from_enemy` is unchanged.

3.9.5 Item factories (@register_item)

Each factory in `combined.py` is a `Callable[[Piece], EffectBundle]` registered via `@register_item(item_id)`. A factory returns an `EffectBundle` containing:

- **modifiers:** flat stat deltas (`Modifier(stat, delta, Lifetime.COMBAT)`).
- **hooks:** `list[Hook]` -- each `Hook(event_name, fn, scope)` is registered on the `EventBus`; closures capturing `owner: Piece` for per-instance state (one-shot flags, cadence counters).

3.9.5.1 Raw components (8)

ID	Grants
<code>fang</code>	+12% STR (multiply)
<code>keen_claw</code>	+15% crit chance (additive)
<code>talon</code>	+12% AS (<code>attack_speed</code> + <code>milli_AS</code> paired)
<code>wardpelt</code>	+14% MR
<code>stoneplate</code>	+14% Armor
<code>old_hide</code>	+12% HP
<code>heartseed</code>	+12% INT
<code>springtear</code>	<code>x1.15 mana_regen</code> (Modifier) + <code>on_combat_start</code> hook: +100_000 flat starting mana (~=1/3 of default cost, V.48)

`springtear` grants `mana_regen` (the cast-rate knob) plus `start_mana` via an `on_combat_start` hook (mana is per-ActiveSlot, not in `Piece.base_stats`). **Mana items NEVER reduce mana_cost** (V.48, T.29c -- kills negative-cost stacking, B.21); they grant `mana_regen` or `start_mana` only.

All modifiers use "mul" (multiplicative) or "add" (additive) operation via `Modifier(stat, op, delta, Lifetime.COMBAT, source)`. The multipliers in the factories represent the scale factor (e.g. `1.12` = `x1.12` the base stat).

3.9.5.2 Combined items (16-core cut)

ID	Components	Key effects
<code>apex_fang</code>	<code>fang</code> + <code>fang</code>	<code>x1.24 STR</code> ; on-kill grants +5% of current STR (compounding add)
<code>tempest_talons</code>	<code>talon</code> + <code>talon</code>	<code>x1.24 AS</code> ; each auto-hit adds +0.5% of current AS (compounding ramp)
<code>worldroot_bloom</code>	<code>heartseed</code> + <code>heartseed</code>	<code>x1.30 INT</code> (pure stat stick)

ID	Components	Key effects
deepwell	springtear + springtear	x1.30 <code>mana_regen</code> + +200_000 flat starting mana; combat-start barrier (15% holder max HP) on lowest-HP ally (support); after first cast, refunds 50% <code>mana_cost</code> (clamp <code>max_mana</code>) per cast (V.48)
mammoth_hide	old_hide + old_hide	x1.24 HP; every 2 s heals holder + adjacent allies 2% max HP (team regen aura $\sim$ 1%/s, ungated)
bramble_carapace	stoneplate + stoneplate	x1.28 Armor; flat 80 magic retaliate + grievous (halved healing, 2 s) to melee attackers (thorns, flat by design)
mistward_shroud	wardpelt + wardpelt	x1.28 RES; regenerates 1% max HP every second (self only)
perfect_predator	keen_claw + keen_claw	+30% crit chance; critical hits deal +25% bonus damage (ITEM_PROC)
bloodthorn_briar	fang + heartseed	x1.12 STR, x1.12 INT; heals holder for 18% of all damage dealt
wildfury_lass	talon + heartseed	x1.12 AS, x1.12 INT; each auto adds +1% current AS; every 5th auto triggers a free cast
everbloom_staff	heartseed + springtear	x1.12 INT, x1.15 <code>mana_regen</code> , +100_000 flat starting mana; INT grows +1% per 2 s while alive (V.48)
witherbloom_censer	heartseed + old_hide	x1.12 INT, x1.12 HP; autos apply burn (3 s) + RES x0.80 sunder + grievous (halved healing) (single refreshing instances)
stormglass_totem	heartseed + wardpelt	x1.12 INT, x1.14 RES; when a nearby enemy casts (radius 5), zaps them for INTx0.50 magic damage
spellfang_crown	heartseed + keen_claw	x1.12 INT, +15% crit chance; abilities can crit (<code>ability_can_crit</code> set on <code>on_combat_start</code>)
living_bulwark	old_hide + stoneplate	x1.12 HP, x1.14 Armor; combat-start +18% Armor aura to adjacent allies (support anchor)
splitwind_talons	talon + wardpelt	x1.12 AS, x1.14 RES; each auto strikes nearest second enemy within range 2 for 50% dmg + applies Slow (soft CC) to both

Hook guards: `on_damage_dealt` hooks check `ev.tag == SourceTag.ITEM_PROC` to avoid triggering on bonus damage from the same item (preventing infinite loops).

`ability_can_crit` is set in an `on_combat_start` hook (matching the `ability_crit()` idiom in `traits/mechanics.py:268-273`).

milli_AS pairing: every item that modifies `attack_speed` also modifies `milli_AS` (V.34 / sort-order invariant).

3.9.6 REWARD-node drops (encounter.py)

```
CH_REWARD: Final[int] = 8 # seed channel for reward loot
```

```
@dataclass
class RewardLoot:
```

```
item_ids: list[str]
```

```
def generate_reward_loot(run_seed: int, node_index: int) -> RewardLoot:
```

Deterministic drop table (derives from (run_seed, node_index, CH_REWARD)):

Roll	Probability	Outcome
< 0.60	60%	1 base component
< 0.85	25%	1 core combined item
else	15%	2 base components

All item IDs returned are guaranteed members of `BASE_COMPONENTS` U `ITEM_REGISTRY`.

3.9.7 Combined items (remaining 20 -- T.29b)

`combined.py` (T.29b block). Hook items use the closure/cadence/`secs()` patterns; all magnitudes first-pass, judged against combat scale (autos ~5 s, HP 600-1500).

ID	Components	Key effects
<code>huntress_talon</code>	fang + talon	+12% STR/AS; autos apply stacking poison (3 s)
<code>relentless_spear</code>	fang + springtear	+12% STR, +15% MR; +30k mana per auto
<code>titanbone_charm</code>	fang + old_hide	+12% STR/HP; +0.4% STR per attack/hit, barrier (15% HP) at 12 stacks
<code>beastheart_gauntlet</code>	fang + stoneplate	+12% STR, +14% Armor; barrier (25% HP) first time below 35%
<code>twinclaw_pact</code>	fang + wardpelt	+12% STR, +14% RES; alternates +50% bonus hit / 30% heal
<code>giantsbane</code>	fang + keen_claw	+12% STR, +15% Crit; autos +4% target max HP magic (anti-tank)
<code>stormscale_quiver</code>	talon + springtear	+12% AS, +15% MR; every 4th auto chains lightning to <=3 enemies
<code>quickpelt_harness</code>	talon + old_hide	+12% AS/HP; first hard-CC -> cleanse + 3 s CC-immune
<code>sundertalon</code>	talon + stoneplate	+12% AS, +14% Armor; autos shred target Armor x0.82 (3 s)
<code>stalkerclaw</code>	talon + keen_claw	+14% AS, +15% Crit (clean crit stat stick)
<code>stoneward_idol</code>	heartseed + stoneplate	+14% INT, +16% Armor (durable backline caster)
<code>sapwood_aegis</code>	springtear + old_hide	+15% MR, +12% HP; start shield (20% HP) -> INT burst on break
<code>wardens_dewstone</code>	springtear + stoneplate	+15% MR, +14% Armor; +15% MR aura to adjacent allies
<code>seasonward_charm</code>	springtear + wardpelt	+15% MR, +14% RES; adaptive +20% Armor/RES vs recent damage type
<code>dewclaw_fetish</code>	springtear + keen_claw	+15% MR, +15% Crit (cast-cycling crit carry)
<code>spiritbark_hide</code>	old_hide + wardpelt	+12% HP, +16% RES (anti-magic brick)

ID	Components	Key effects
gorehide_wrap	old_hide + keen_claw	+14% HP, +15% Crit (fragile crit carry survivability)
greatward_carapace	stoneplate + wardpelt	+14% Armor/RES; +4% Armor & RES per living enemy at start
edge_of_stone	stoneplate + keen_claw	+16% Armor, +15% Crit (bruiser-carry hybrid)
hexward_claw	wardpelt + keen_claw	+16% RES, +15% Crit (crit that survives magic burst)

3.9.8 Emblems (6 -- emblems.py, T.29b)

Each carries `granted_traits=["<Kinship>"]` + an 8% flavour stat; applied at the item step (§10.1 step 2.5) **before** `_resolve_traits`, so the wearer counts toward that Kinship breakpoint. Crafted via `combine(spirit_gem, component)`.

Emblem	Kinship	Stat	Crafted from
beast_emblem	Beast	+8% STR	spirit_gem + fang
skyborn_emblem	Skyborn	+8% AS	spirit_gem + talon
scaled_emblem	Scaled	+8% Armor	spirit_gem + stoneplate
tidekin_emblem	Tidekin	+8% MR	spirit_gem + springtear
swarm_emblem	Swarm	+8% HP	spirit_gem + old_hide
spirit_emblem	Spirit	+8% INT	spirit_gem + heartseed

3.9.9 Special items -- run-actions (special.py, T.29b)

Never enter combat (V.24) -- `RUN_ACTION_REGISTRY`, operate on Run only. Spirit Gem is the 6th special, handled inline by `combine()` (crafting, not a run-action).

Run-action	id	Effect on Run
Wildwood Reforging Stone	reforger	Swap one component of a combined item (deterministic next) + recombine
Unbinding Totem	unbinding_totem	Strip a champ's items -> bench inventory, decomposed to components
Echo Acorn	echo_acorn	Add a bench copy of a champion (+ <code>champion_copies</code> , T.22 levelling)
Glimmerdust	glimmerdust	Upgrade item -> <code>heartwood</code> : version (D.21 MVP: x1.5 modifiers at equip)
Reclaimer's Cache	reclaimers_cache	Salvage base components -> 10 Amber each

Heartwood (`heartwood:<id>`) is scaled at equip by `loadout._heartwood_scale` (mul/add modifiers x1.5; hooks/procs untouched -- D.21 MVP). `decompose()` reverses any item to base components (used by `unbind/reforge`).

CLI: `sim_run --interactive` opens a prep shell (`combine/equip/reforge/ unbind/echo/glimmer/salvage`) over a Run before the route walk.

Status: **LIVING** -- conventions for authoring champion/enemy kits + axes. Must match the formulas in `src/game/combat/context.py`, `src/game/scaling.py`, `src/game/content.py`. Audited by `/check` (cited constants must hold; the prose rules are review-enforced). **Scope:** the balance/identity anti-patterns to watch when writing or re-axising a kit. **Reconciled:** 2026-06-16 @ T.36 roster rebalance (kings + the 3 caster->auto flips).

3.10 Kit design conventions -- faults to watch

Hard-won from the T.36 Primordial rework + the 12-piece distribution re-axis. Each rule below was paid for by a real misfit caught mid-design. Read before authoring or re-axising.

3.10.1 Identity & role

1. **The Calling fixes the *playstyle*; the stat stays flexible.** Cast-Callings (Channeler / Mystic / Multicaster / Warden / Mender) -> **ability**. Auto-Callings (Hunter / Skirmisher / Stalker / Bruiser) -> **auto**. Guardian = tank, *leans* cast but soft. **Never put a Channeler in an auto cell or a Hunter in an ability cell** -- the kit reads wrong even when the numbers are fine. (T.36 caught this in 3 of 6 kings + 4 of 12 distribution pieces; the original plan draft had them backwards.) When a target cell needs a playstyle the Calling fights, the cheap fix is a **minimal, lore-natural Calling tweak** (add an auto-Calling), not a forced playstyle.
2. **Hold intent -> preserve the role. Flex stat + playstyle freely.** `classify_role(stat, reach, durability, playstyle, speed, intent)` -- intent (with durability) is the dominant role lever. Changing stat/playstyle reshapes *how* a piece fights; changing intent silently changes *what role it is*. A Hunter at `intent=utility` is still a **support** that happens to auto-attack -- not all auto-Callings are dealers. Verify `build_role_code` before/after any re-axis; if the role moved and you didn't mean it to, you changed intent by accident.
3. **Don't erase a piece's soul when you re-axis it -- relocate it.** `dusk_bat` (debuff-support) flipped to `str/auto` kept its blind by moving it *onto the autos*. `phantom_lynx` kept its pen/ghost identity by moving "ignore defense" onto a true-damage empowered auto. The mechanic moves to the new playstyle; the fantasy stays.

3.10.2 Coefficients & scaling

4. **str/ability: the ability is the main value; the STR coeff sits *below* the INT baseline.** The free auto-attack tagalong already pays STR (autos are $1.0\text{-STR} + 0.25\text{-INT}$, `context.py:~409`). Parity vs the INT coeff it replaces: `coeff_str ~= coeff_int - 0.667 - (autos_per_cast)` -> a ranged caster lands ~1.1-1.7; AoE+CC sits at the low end (the CC is the payoff). **Not** the old "ability empowers autos" steroid. (Mournhollow's naive STR-2.7 was ~2x over.) **The free-auto subsidy is universal -- it applies to casters and supports too, not just carries.** Any STR- or hybrid-*stat* piece has live autos (1.0-STR base) that out-chip an INT peer's dead 0.2-STR autos *for free*, even at a caster's low `attack_speed` (measured: a hybrid-stat support out-chips an INT support by ~5-18 DPS from the stat alone). So a STR/hybrid-stat **support's** ability coeff must also be discounted vs its INT peers, or it out-budgets them (T.36b: `dawnwisp` flipped `int->str` support with its heal discounted `INT-4.55->STR-3.6`, ~0.8x for the subsidy). The naive `int/ability -> str/ability` stat-swap that

keeps the old INT coeff (mirewarden, hollow_elk had INT-3.3-3.9) is the same landmine -- drop the coeff hard on the swap.

4b. **When a re-axis produces a role that *misrepresents* the piece, suspect a taxonomy hole -- don't force a misfit.** A ranged playstyle-hybrid damage dealer (casts *and* autos) classified as marksman because `classify_role` lumps hybrid-playstyle with auto (`content.py:184`). That was a real hole -- the ranged analog of `spellblade` (which is *stat*-hybrid). Fix = a new role (`Spellslinger`), not a contorted axis. Check `build_role_code/classify_role` on every re-axis; an off-reading role is a signal.

5. ***/auto (str/auto, int/auto, hybrid/auto): the autos must carry -- fold the active's payoff *into* an auto.** Patterns: on-hit proc (int/auto = on-hit-INT, no STR), empower-next-auto (Yorick-style), discharge-on-auto. If the piece "doesn't care about its autos," it isn't really an auto piece. (phantom_lynx's whole active resolves through an auto; granite_gorilla discharges its bank through autos.)
6. **Beware the hidden multiplier: per-event scaling x event count.** `charge += STR-k` *per blow* over N blows = an effective coeff of k-N, not k. A tank eats many hits -> N is large -> a normal-looking k becomes a hypercarry coeff, and stat items then scale it. **Fix:** keep k low *and* hard-cap the accumulator (`charge <= STR-1.5`) so the N multiplier can't run -- leaving linear stat scaling. Same family as the Aurion bug (`+1 primary/tick` -> ~600% over a fight); both are unbounded per-tick/ per-event ramps. Always bound a ramp by stacks or a cap.
7. **Size penetration against the actual resistance ceiling.** Penetration is a **global attacker stat** (`penetration / penetration_pct`, `context.py:256-257`), *not* per-hit -- you cannot scope it to one damage instance. Flat pen subtracts from mitigation, so it **zeroes any target whose res < pen**. Max resistance in the roster ~ = **359** (T7 tanky_arm), typical big tank ~286, `MITIGATION_CONSTANT=100`. Size flat pen so it shreds tanks *partially* and never blankets the midfield (phantom's INT-0.12 peaks ~49 ~ = 14% of max res; INT-0.3 zeroed everything under 123).
8. **True damage is premium -> lower its coeff than a magic hit.** `SourceTag.TRUE` bypasses all mitigation (`context.py:253-254`). Prefer it over penetration for an "ignore defense" finisher, but price it below a mitigated nuke.

3.10.3 Retaliation, sustain, persistence

9. **Never tie retaliation to a flat/% of *incoming* damage when HP pools are asymmetric.** A tank reflecting %-of-hit returns trivial damage to itself but *lethal* chip to a squishy -- worst case **a squishy dies faster attacking the tank than the tank dies**. Punishes the wrong axis. Use a banked-and-discharged model directed at the retaliator's *own* target, capped, scaled by the retaliator's own stat (not enemy damage). (granite_gorilla.)
10. **Scope a diver's sustain to its own commitment, not a passive omni-stat.** A squishy auto-carry/swashbuckler gets lifesteal tied to its burst/true-strike (so it must commit and land to be rewarded), never free lifesteal on every hit. (phantom_lynx reaps only off the empowered true-damage auto.)
11. **Piece stat-stacking is in-combat only; cross-Run permastacking is augment-exclusive.** Combat is a pure function (V.2) -- all piece runtime state rebuilds per `resolve_combat`, so nothing persists across battles by construction. Write blurbs as "until

end of battle,” never “permanently” (which mislabels an in-combat ramp).

3.10.4 Process & verification

12. **The analytic DPS / HP-DPS proxy is a smoke-detector, not a scale.** Single-target DPS *understates* AoE+CC casters (no AoE multiply, no CC value) and auto-carries with uncounted procs/steroids. Use it to catch a piece sitting wildly off-budget (it caught Mournhollow at ~147k). The real gate is the $V.33 \pm 10\%$ HP-DPS proxy
 - `tools/simulation/stat_edge.py` teamfight sims.
13. **”power” is reserved.** `scaling.power(T,L) = 2^((T-1)/3 + triplings(L))` is the abstract power scalar. The survivabilityxdamage worth proxy is **HP-DPS** -- never call it ”power.”
14. **Determinism is non-negotiable (V.2/V.14).** Every ”every Nth” / ”chance” / ramp uses a deterministic cadence counter (like `crit_counter`), never RNG -- sims must stay byte-identical.
15. **When reshuffling axis assignments, preserve the *destination cell multiset*.** The `statxplaystyle` grid marginals are the contract, not which piece sits where. Reassign freely to honor Callings as long as the multiset of target cells is unchanged -- the grid still lands exactly. (Both T.36a kings and T.36b distribution were corrected this way with zero grid impact.)

3.11 Rosters -- champions, enemies, bosses

Status: LIVING -- source of truth is `src/game/content.py` + `bosses/data.py`. Audited by `/check` (counts). **Scope:** the live roster invariants, the `def->model` build, and where the real data lives. **Reconciled:** 2026-06-05.

This is a **thin pointer**: `stat` blocks, names, and lore live in code + the frozen catalogs (`docs/design/content/champion_roster.md`, `enemy_roster.md`, `boss_roster.md`). Here = the invariants and the build path.

3.11.1 Counts (verified; `/check` re-checks against code)

- **60 champions** -- `len(CHAMPION_ROSTER)`; tiers 1-10 x 6 affinities (`clear/cloudy/mist/rain/snow/thunder`).
- **60 enemies** -- `len(ENEMY_ROSTER)`.
- **6 bosses** -- `len(BOSS_DEFS)`.

3.11.2 Source of truth (code, not this doc)

- `content.py` -- authored tuples `_CHAMPION_DEFS: tuple[ChampionDef, ...]` and `_ENEMY_DEFS: tuple[EnemyDef, ...]` are the design data. They're built into the runtime model dicts:
 - `CHAMPION_ROSTER: dict[str, Champion]`, `ENEMY_ROSTER: dict[str, Enemy]`, `ENEMY_DEF_BY_ID: dict[str, EnemyDef]` (formation/encounter read the defs).
 - Accessors `get_champion(id)` / `get_enemy(id)`.
- `bosses/data.py` -- `BOSS_DEFS: dict[str, BossDef]` (kit, map effect, spawn).

3.11.3 Role taxonomy

A piece's `role/role_code` are **derived**, not authored: `classify_role(...)` maps the design axes to a role title and `build_role_code(...)` to the role code (see SPEC §V.31-V.33, the T.32 role/intent system). `CALLING_TAGS` is the frozen trait-tag vocabulary `Champion.traits` draws from (see [traits.md](#)).

`Champion` and `Enemy` share the same stat block + ability framework + damage math -- they differ only in operation (champions are drafted/bought/levelled, carry traits). The combat entity built from either is the runtime `Piece` (see [../systems/combat.md](#)).

3.11.4 Stat-axis balance (T.35b, #42 Finding B / B.20)

The `stat` axis x `durability` jointly set a unit's primary. T.35b re-tuned two axis tables in `content.py` so a primary-stat **tank no longer rivals an assassin's primary** (was: 1.8-str-axis x 0.55-tanky $\approx$ 0.99 $\approx$ a bruiser):

- `_DURABILITY` tanky_hp / tanky_arm strength/intelligence **0.55** -> **0.42**.
- `_INTENT` damage strength/intelligence **1.08** -> **1.14**, utility **0.94** -> **0.87** (defensive stats compensate so the V.33 HP-DPS proxy stays in [0.90,1.10]: damage 1.075, utility 0.947). Example: Coral Colossus STR 92->65, Duskstep Marten INT 127->134.

Dead-INT carriers fixed: every int/hybrid unit now reads INT in its kit -- per-role INT coefficients added to ~14 carriers' ability outlets (authored as `Magnitudes`, T.35a). Enforced by §V.47 (`test_axis_scaling_alignment`): `stat="int"` must reference INT via a meta `Magnitude`, hybrid references both, `str` is auto-satisfied by the auto-attack. `enemy_steam_engineer` is allowlisted (its INT sizes the turret `SummonSpec`, not a meta outlet).

3.11.5 Axis-distribution rebalance (T.36)

The roster axes were rebalanced to principled marginals (a unified solve -- role is a pure fn of axes, V.32, so the marginals + soft role floors are the target and the role distribution is derived). Live counts (recompute from `_CHAMPION_DEFS` / `_ENEMY_DEFS`; ± 2 of target is in-band):

axis	champions	enemies
stat	str 22 - int 22 - hybrid 16	str 22 - int 22 - hybrid 16
playstyle	auto 24 - ability 24 - hybrid 12	auto 22 - ability 22 - hybrid 16
reach	melee 28 - ranged 32	melee 30 - ranged 30
durability	thp 11 - tarm 6 - squishy 13 - hybrid 30	thp 11 - tarm 7 - squishy 13 - hybrid 29
intent	damage 28 - utility 20 - hybrid 12	damage 29 - utility 19 - hybrid 12

Emergent enemy roles (T.36c -- curated by name/lore, opaque tags V.22): tank 12 - support 11 - spellblade 6 - bruiser 6 - mage 6 - swashbuckler 6 - marksman 5 - assassin 4 - spellslinger 4. New `Spellslinger` role (V.32, T.36b). All roles ≥ 4 . Re-axised kits honor V.46/V.47 (every int/hybrid enemy reads its primary stat; hybrid ability-users read both). Combined champ-vs-enemy **sim balance-validation DONE** [2026-06-17] (`tools/simulation/stat_edge.py`, team sims 2-5v5 x all weathers, `results/stat_edge_t36c.csv` n=8000 + `iterate_t36d.csv` n=1500): **zero champs over the $|wr_delta| > 0.10$ contract bar** after a 5-champ tune (`mournhollow/veilfang_wolf/ember_salamander` trimmed; `aurion/will_o_fawn` buffed -- `champions.py`). 44/60 inside ± 0.05 ; the ± 0.05 stretch on 3 stubborn over-performers (`ember/mournhollow/veilfang` $\sim +0.085$, within

n=1500 noise) is deferred to the full random-vs-random power sim. **D.25 reframe:** the residual STR-ability edge (+0.035) is STR-ability *over*-budget (free auto-tagalong 1-STR+0.25-INT), not INT-ability under -- no further global INT bump warranted.

3.11.6 Invariants

- Counts above hold (per the §T content budget); /check recomputes them.
- Every `active_ability/passive_ability` id on a def resolves in its registry -- see [abilities.md](#) (CI-guarded).
- §V.47 axis<->scaling: `int/hybrid` units reference INT in their kit (no dead INT).

3.11.7 File map

Concern	Symbol
Champion design data -> models	<code>content.py</code> (<code>_CHAMPION_DEFS</code> , <code>ChampionDef</code> , <code>CHAMPION_ROSTER</code> , <code>get_champion</code>)
Enemy design data -> models	<code>content.py</code> (<code>_ENEMY_DEFS</code> , <code>EnemyDef</code> , <code>ENEMY_ROSTER</code> , <code>ENEMY_DEF_BY_ID</code> , <code>get_enemy</code>)
Role derivation	<code>content.py</code> (<code>classify_role</code> , <code>build_role_code</code> , <code>CALLING_TAGS</code>)
Bosses	<code>bosses/data.py</code> (<code>BossDef</code> , <code>BOSS_DEFS</code>)

3.12 Abilities & passives -- id resolution

Status: **LIVING** -- must match `src/game/abilities/` + `registries.py`. Audited by /check. **Scope:** how a roster ability/passive id becomes a runtime handler, and the resolution guarantee (T.30). **Reconciled:** 2026-06-05.

Citations by symbol, not line. The *kits* (what each ability does, by lore) live in the frozen `docs/design/content/ABILITY_CATALOG_CHAMPIONS.md` / `ABILITY_CATALOG_ENEMIES.md`. The framework they plug into is [../systems/effects.md](#).

3.12.1 Id convention

A Champion/Enemy carries `active_ability` / `passive_ability` **string ids**, authored as prefixed ids in `content.py` (`active_ability` = `f"{id}.active"`, e.g. `champ_torrent_heron.active`). Handlers register under those exact ids -- the T.30 fix was aligning the two (previously short ids vs prefixed ids meant 0/240 resolved and every piece ran the generic fallback).

3.12.2 Registration

Handlers live in `abilities/champions.py`, `abilities/enemies.py`, `abilities/bosses.py` (+ `abilities/reference.py` for shared/example kits) and register at import via decorators from `registries.py`:

- `@register_active(ability_id)` -> `ABILITY_REGISTRY`
- `@register_passive(passive_id)` -> `PASSIVE_REGISTRY`
- `register_active_simple(id, SimpleActive(...))` -- declarative single-target spec abilities (no hand-written handler).

`compile_loadout` imports the `abilities` package, which triggers every decorator, populating the registries before any combat.

3.12.3 Counts (verified; /check re-checks)

- `len(ABILITY_REGISTRY) = 144`, `len(PASSIVE_REGISTRY) = 147` (shared handlers serve multiple roster ids; the guarantee below is per-roster-id, not per-handler).

3.12.4 The resolution guarantee (CI-guarded)

Every ability/passive id referenced by the roster must resolve to a registered handler -- enforced by a guard test (SPEC §V.15/§V.17). An unregistered id is a build failure, not a silent generic-fallback. The loop's unregistered-ability path (`engine._resolve_action`) is only a defensive fallback, not the norm.

3.12.5 Ability descriptions (T.34)

A parallel registry `ABILITY_META: dict[str, AbilityMeta]` (in `registries.py`) gives each roster ability id a tooltip. `AbilityMeta(name, kind, blurb, terms, clauses, tags)` is presentation metadata; `ability_text.render(meta, source) -> RenderedAbility(name, text, formula, tags)` is **pure** (no Flet/I-O, extends V.1) and reads live numbers via `source.stat(name)`. The same call serves both contexts -- a base Champion/Enemy (roster sheet, via the `.stat()` field adapters) and a live **Piece** (combat, with modifiers; **bosses always via the compiled Piece**). `render_for(id, source)` is the dict-lookup convenience.

Each scaling term renders its coefficients as **percentages of the source stat** so players read what a number scales from: `formula` lines read `550 = 100 + 150% STR + 150% INT` (`STR 1.5x150`, `INT 1.5x150`) -- the trailing note shows each term's contribution math (`coeffxstat`), not the bare stat value, so the headline number is fully traceable -- and `text` carries a compact inline suffix beside the rendered total (...`550 magic damage. (100 +150% STR +150% INT)`). Pure-flat / no-scaling terms (buffs) add no suffix.

Source-of-truth B (V.38 + V.46): every stat-scaled outlet a handler computes lives **once** in a **Magnitude** the handler reads via `term.eval(...)` -- the tooltip renders the same object, so tooltip and combat numbers cannot drift. T.35a promoted `ScalingTerm` into a **closed Magnitude family** (modeled on Unreal GAS's `EGameplayEffectModifierMagnitude`), so there is **no** free inline handler math anymore -- the old "Tier-B stays inline + prose" carve-out is gone. The four kinds (all in `registries.py`, all pure/RNG-free, all self-describing via `eval/render_formula/render_inline/render_token`):

kind	shape	absorbs
<code>ScalingTerm</code>	<code>base + Σ source.stat-coeff</code> (delegates to <code>_eval_scaling</code>)	linear damage/heal/buff/shred
<code>PctResource</code>	<code>getattr(obj, resource)-pct</code> , <code>of="self" "target"</code>	%-of-max-HP heals (reads <code>.max_hp</code> directly -- <code>Piece.stat("max_hp")</code> is 0, see <code>effects.compute_stat</code>)
<code>MaxOfTerm</code>	<code>base + max(source.stat(s)...)-coeff</code>	<code>max(STR,INT)</code> outlets (non-linear)
<code>SetByCaller</code>	<code>base + caller[key]-coeff</code> (handler injects the runtime value)	per-stack / runtime-count outlets

Clause carries an optional {token} template + its own terms, so a Tier-B scaler's prose number is filled from the same Magnitude the handler reads (A1) -- e.g. Hierarch's Grants Armor ({armor}).... SummonSpec holds a summon's statline as Magnitude fractions + flat literals (eval(owner) -> base_stats), so summon stats are introspectable, not inline. **A2 guard (test_no_orphan_stat_reads, V.46)**: an AST walk fails the build if any handler reads .stat()/.max_hp outside a Magnitude on its meta -- except ids on _PROSE_ALLOWLIST (flat max_hp += growth: champ_snowpelt_cub.passive, champ_glacierback_mammoth.passive, enemy_levyman.passive). The conversion was **byte-identical** (sim digest unmoved, V.2/V.14); only the rendered formula/text of the 15 converted Tier-B abilities gained their scaling lines.

Tick -> seconds (V.39): TICKS_PER_SECOND = 100 in ability_text.py is the single source for the 100 ticks = 1 second display convention. Mechanics stay ticks-only; only ability_text.render (and ui/, which imports the constant from here) converts. A coverage guard + golden formula snapshot (tests/game/ability_formulas.snapshot.json) pin every rendered id; regenerate with UPDATE_ABILITY_SNAPSHOT=1 uv run pytest tests/game/test_ability_text.py.

Meta coverage spans **all 285 roster ability ids** (120 champion T.34a + 120 enemy T.34b + 36 boss T.34c = 276 base, + 9 T.29d multicaster .active2 secondaries -- all done). tools/export_roster.py serializes the champion/enemy (and optional boss) rosters with rendered descriptions to JSON.

3.12.6 File map

Concern	Symbol
Champion kits	abilities/champions.py
Enemy kits	abilities/enemies.py
Boss kits (2-phase)	abilities/bosses.py
Shared/declarative	abilities/reference.py, registries.register_active_simple
Registries + decorators	registries.py (ABILITY_REGISTRY, PASSIVE_REGISTRY, @register_active/@register_passive)
Ability descriptions (T.34/T.35a)	registries.py (ABILITY_META, Magnitude family: ScalingTerm/PctResource/MaxOfTerm/SetByCaller, Clause w/ template+terms, SummonSpec, AbilityMeta), ability_text.py (render pure per-kind dispatch, render_for, TICKS_PER_SECOND)
id source on the model	content.py (active_ability/passive_ability)

3.13 Traits -- kinships, callings, breakpoints

Status: LIVING -- must match src/game/traits/ + content.py trait vocab. Audited by /check. **Reconciled:** 2026-06-16 (T.28d + T.29d Multicaster).

[stub] **PARTIAL** -- T.28a/b/c/d built the framework + stat packs + the full reachable combat surface. T.28b combat primitives (second-wind, tidal HoT, enrage, time-ramp, dodge, hexproof opener; kiting, backline, taunt, revive). **T.28c (done)** added the rider batch over existing ctx: Hunter bonus-auto/empowered/cleave/team-aura, Mystic ability_can_crit+splash, Guardian start+periodic shields, Bruiser lifesteal, Skirmisher @8 team ramp, Stalker hi-HP-bonus/mana-on-kill/hexproof-after-kill, Chan-neler free-cast/first-cast-twice, Warden cast-shield+team-opener, Trickster slow+taunt-

on-cast+mana-aura, Mender heal-splash/overheal-shield, Spirit echo, Swarm on-death chitin spawn. **T.28d (done):** renamed `untargetable->hexproof` + fixed single-target acquisition to honor the gate (AoE still hits; `pierces_hexproof` bypass -- V.40, B.15); 5 affinity @10 riders (Galvanized crit-arc, Frostbound chill-attackers, Stormfed mana-haste [stat], Shrouded longer hexproof-opener, Overcast burst-reduction) + Sunlit premium-stat @10 pack; Scaled @5 hard-CC immunity + @8 favorable weather override; Spirit @8 reduced-potency echo (0.6x) + mana-haste + hexproof pierce; fold-ins Beast @4/@6 strength-ramp, Skyborn @3 kite-reward, Tidekin @3 ally-heal. **Deferred to T.31 (D.20):** 6 Primordial @1 signature mechanics (legendaries already carry full T.30 kits; signatures unreachable until the unlock augments)

- Primordial @3 tier-up. Primordial @1 ships as its stat pack only. Design (frozen): [docs/design/content/trait_catalog.md](https://docs.design/content/trait_catalog.md) v2.1.

3.13.1 Where it lives

- `game/traits/types.py` -- `TraitScope` (`PER_TRAIT_PIECE/TEAM_WIDE`), `TraitBreakpoint`(count, scope, bundle_factory), `DynamicThreshold` (callable(team, board_cap) -> int).
- `game/traits/_packs.py` -- `stat_pack_bundle` (mul/add Modifiers; an `attack_speed` mul moves tie-order on its own now that AS is a float, V.34 / T.29-pre -- no `milli_AS` rider) + `define_trait` shorthand (registers a trait from (count, scope, muls, adds) rungs). `define_trait` also records each rung's raw (label, muls, adds) in `TRAIT_STAT_PACKS` so the description layer can derive a rung's stat line from the same numbers the bundle applies (T.41b, V.79).
- `game/traits/{affinities,kinships,callings}.py` -- the 25 trait factories (declarative stat-pack rungs). Imported by `traits/__init__`, which registers them into `TRAIT_REGISTRY` as a side effect.
- `game/traits/__init__.py` -- `affinity_trait` (affinity -> derived tag), `_resolve_traits`, `resolve_and_apply_traits`.
- `game/registries.py` -- `TRAIT_REGISTRY` + `@register_trait`.
- `content.py` -- `KINSHIP_TAGS` (6) / `CALLING_TAGS` (13) / `ALL_TRAIT_TAGS`; `Champion.traits`.

3.13.2 The 25 traits

- **6 Kinships** (`KINSHIP_TAGS`): Beast, Spirit, Skyborn, Scaled, Tidekin, Swarm.
- **6 Affinities** (derived from `affinity`, never stored): Sunlit/Overcast/ Shrouded/Stormfed/Frostbound/Galvan = Clear/Cloudy/Mist/Rain/Snow/Thunder.
- **13 Callings** (`CALLING_TAGS`): Hunter, Guardian, Mystic, Warden, Stalker, Bruiser, Skirmisher, Channeler, Mender, Trickster, Packmate, Primordial, **Multicaster** (T.29d -- quick-caster on multi-slot champs; @2/3/4 per-trait, `cast_momentum` stacks `attack_speed+mana_regen` per cast; ~6-carrier pool).

3.13.3 Resolution (in `compile_loadout`, step 3)

1. After weather (step 2), before passives (step 7).
2. `_resolve_traits(team, board_cap)` counts **unique champion ids** per tag (V.21), incl. each piece's **derived affinity tag**; finds the highest cleared rung (dynamic thresholds resolved against `board_cap == len(team)`).
3. Applies each cleared rung's bundle to its targets (`PER_TRAIT_PIECE` -> carriers; `TEAM_WIDE` -> whole team). **Player team only -- enemies never light up** (V.22).

4. **HP re-sync:** the engine reads `piece.max_hp/hp` as cached fields, so after applying `hp-mul` modifiers the function recomputes `max_hp = hp = piece.stat("hp")` (pieces start each combat at full HP).
5. Returns `trait_activations: [(trait_id, count, threshold), ...]` (sorted), surfaced through `compile_loadout's` 3-tuple return -> `BattleResult`.

Pure + RNG-free -> replay-stable (V.21). Counting is at loadout; mid-combat spawns/revives never raise a count.

3.13.4 Breakpoint shapes (apex = `min(pool, cap)`, V.37)

Ladders are single-step-leaning with @1 entries on supports/casters/kilters; see the catalog for per-trait rungs. **T.28a implements the stat-pack portion of every rung**; mechanic riders layer onto the same trait ids in b/c. Packmate's apex is a **dynamic @full-board** threshold (== fielded board size).

3.13.4.1 Cumulative rungs (load-bearing) -- V.41

Resolution applies **only the highest cleared rung's bundle** (one `TraitBreakpoint`, not a union). Stat magnitudes are authored as the **total at that rung** (they replace, not stack); mechanic riders persist **only if each higher rung re-lists them**. **V.41** guards this: `test_trait_rungs_are_cumulative_for_mechanics` asserts every mechanic fingerprint at rung N reappears at rungs > N -- sole exception, carrier-movement (kiting/backline) at a `TEAM_WIDE` apex. So each rung **re-includes every mechanic it should still grant** -- e.g. Skirmisher carries `time_ramp` on @2/@3/@4/@5/@8 and `dodge` on @4/@5/@8. Forgetting to re-include silently drops a lower mechanic at the higher count. **Apex exception:** carrier-movement mechanics (kiting, backline) are NOT re-applied at a TEAM apex -- applying them team-wide would make every ally kite/seek; apex trades the few-carrier identity for a team buff. Caster/role-gated riders (echo, splash, lifesteal, shields, on-death spawn) ARE team-safe (no-op for the wrong role) and stay; the Swarm on-death spawn is explicitly `trait="Swarm"`-guarded so a TEAM apply only spawns for Swarm pieces.

3.13.5 Combat primitives (T.28b) -- `game/traits/mechanics.py` + engine

All deterministic (cadence counters / HP thresholds / geometry, never RNG -- V.2/V.14/V.37). A rung's optional 5th tuple element is a list of builders (`owner, source_id`) -> `list[Hook]` appended to its bundle (`_packs.define_trait`).

Hook riders (ride the event bus, no engine edits):

- `second_wind` (Primordial @2) -- `on_damage_taken`; first drop below 60% HP -> `grant_barrier(0.4-max_hp, 1200t)` once. Reuses V.28 barriers (decay = expiry).
- `tidal_hot` (Tidekin @5/@8) -- `on_tick` cadence `ctx.heal` of the carrier.
- `enrage` (Beast @8) -- `on_damage_taken`; one-shot AS+STR burst below 25% HP.
- `time_ramp` (Skirmisher @2) -- `on_tick` stacking AS mul to a cap.
- `dodge` (Skirmisher @4) -- `on_damage_pre` reducing; every Nth basic -> 0 (engine floors final to 1, so a near-total mitigation).
- `hexproof_opener` (Spirit @5, Shrouded @10) -- `on_combat_start` applies `hexproof`.

Engine-behaviour arms (set a flag/status the engine reads):

- **kiting** (Skyborn @2) -- `on_combat_start` sets `Piece.is_kiter`; melee (base range ≤ 1) also get +1 `attack_range` (no double vs @5's flat +1). Engine `_kite_step` (movement): retreat one hex from a **single** adjacent melee threat that strictly increases distance while keeping ≥ 1 enemy attackable; plant when swarmed (≥ 2 melee), cornered, or no target.
- **backline_seeker** (Stalker @2) -- sets `Piece.seeks_backline`; engine paths to the deepest enemy column (`_backline_subset`) and `_select_target` prefers it. No teleport.
- **revive_first_ally** (Mender @6, TEAM_WIDE) -- `on_death`; the first ally death each combat $\rightarrow$ `ctx.revive(victim, 0.3)`. Once-per-combat shared across all carriers via a flag on `ctx`. The one true revive (V.37); mid-combat revives never raise a trait count.
- **taunt** -- StatusGate-free taunt `StatusDef`; `StatusInstance.source_id` = taunter. Engine `_taunt_target` forces the taunter as target (overrides current/backline) and as the movement goal. Capability only in T.28b -- wired to Trickster casts in T.28c.

`StatusGate.HEXPROOF` (status hexproof, renamed from `untargetable` in T.28d) excludes a piece from **single-target acquisition** -- the engine `_opponents` auto-attack scan **and** every single-target helper in `targeting.py` (`primary_target/lowest_hp_enemy/highest_ap_enemy/random_enemy/furthest_enemy/_closest_enemy`). AoE/untargeted effects (`enemies_in_radius`, `line_targets`, a cast iterating `ctx.enemies_of`) **still hit** -- MTG-hexproof. A piece with `Piece.pierces_hexproof` (Spirit @8) ignores the exclusion. The piece still acts. (V.40, B.15)

3.13.6 Mechanic + apex riders (T.28c) -- `game/traits/mechanics.py`

All hook idioms over the existing `ctx` mutators (no new engine subsystems). Secondary/proc damage uses `SourceTag.ITEM_PROC` -- the existing "follow-up hit" tag that does NOT re-fire `on_attack_landed/on_ability_damage`, so no recursion. Recast/heal-splash guard re-entry with `ctx._in_recast` / `ctx._in_heal_splash`.

- Hunter: `bonus_auto_damage` + `empowered_shot` (cadence) + `cleave` (`on_attack_landed`); @8 TEAM aura.
- Mystic: `ability_crit` (flips `Piece.ability_can_crit`) + `ability_splash` (`on_ability_damage` $\rightarrow$ adjacent enemies).
- Guardian/Warden: `start_shield` (`on_combat_start`), `periodic_shield` (`on_tick` self + adjacent allies), `cast_shield_lowest` (`on_cast`).
- Bruiser/Beast: `attack_lifesteal` (`on_damage_dealt`, basic only).
- Stalker: `high_hp_bonus` (`on_damage_dealt` vs $>60\%$ -HP target), `mana_on_kill` + `hexproof_after_kill` (`on_kill`).
- Channeler/Spirit: `free_cast` (refund every Nth), `recast_first`, `echo_cadence` (re-run via `ctx.cast_ability`, `_in_recast-guarded`).
- Trickster: `slow_on_cast`, `taunt_on_cast` (reuses the T.28b taunt status), `mana_denial_aura` (`on_tick` drains adjacent enemy slot mana).
- Mender: `heal_splash` (`on_heal` $\rightarrow$ lowest-HP other ally), `overheal_shield` (heal to a near-full ally banked as a barrier).
- Swarm: `on_death_spawn` -- a dying Swarm leaves a stat-fraction chitin via the existing summon pattern (`Piece+summon flags+ctx.spawn`); `trait-guarded`.
- Packmate @full-board: the flat stat pack IS the apex payoff (no rider).
- Multicaster (T.29d): `cast_momentum` (`on_cast_complete` $\rightarrow$ each cast stacks +per `attack_speed` mul + `+mr_per` `mana_regen` mul, capped) -- the quick-caster snowball for multi-slot champs.

Cast-triggered riders only fire for **registered** abilities (the unregistered fast-path in `_resolve_action`

deals damage without firing `on_cast`).

3.13.7 Apex riders + arms (T.28d) -- `game/traits/mechanics.py`

All RNG-free. Affinity @10 rungs (PER scope -- at @10 the board is mono-affinity):

- Galvanized: `crit_arc` (`on_damage_dealt + is_crit` -> arc to a neighbour, `ITEM_PROC`).
- Frostbound: `chill_attackers` (`on_damage_taken` -> slow the attacker).
- Overcast: `burst_reduction` (reducing `on_damage_pre`, cap a hit at 25% max-HP).
- Shrouded: `hexproof_opener(300)` (longer team opener).
- Stormfed: `mana-haste` = a fatter `mana_regen` stat in the @10 pack (no hook).
- Sunlit: **stat-only** @10 -- broadened to premium stats (`crit_chance/penetration_pct` flat adds + `mana_regen` mul), deliberately small.

Kinship arms + fold-ins:

- Scaled: `cc_immunity` (@5/@8 -- sets `Piece.cc_immune`; `ctx.apply_status` skips hard-CC, i.e. any `BLOCKS_*`-gated status; slow/DoTs still land). **Signature** -> **carrier-guarded at the @8 TEAM rung** (`cc_immunity(trait="Scaled")`) so the team-wide armor/res pack still blankets the squad but CC-immunity stays a Scaled perk. @8 also marks `Piece.weather_favored` via the **pre-weather pass** `mark_weather_overrides` (loadout step 2-pre, carrier-guarded) -> `_apply_weather_to_piece` uses `WEATHER_BUFF_BASE[weather]` regardless of affinity.
- Spirit @8 (TEAM): `echo_cadence(3, potency=0.6)` goes team-wide (caster-gated, the apex buff) -- reduced-potency echo via transient `ctx._echo_potency`, read in `deal_damage` (damage-only). The `hexproof_opener` (re-included, cumulative) + `pierce_hexproof` are **Spirit signatures** -> **carrier-guarded** (`trait="Spirit"`). `mana_regen` haste (stat) is team-wide.
- Beast @4/@6/@8: `time_ramp(stat="strength")` (re-included up the ladder with enrage @8).
- Skyborn: `kiting` re-included on the @3/@5 PER rungs (cumulative -- was dropped past @2, **B.16**); **not** re-applied at the @8 TEAM apex (movement exception). `kite_reward` (`on_damage_dealt`, bonus vs a target whose `attack_range < gap`) on @3/@5/@8 (team-wide damage buff at the apex).
- Tidekin @3/@5/@8: `ally_tidal` (cadence-heal the lowest-HP ally; distinct from the carrier `tidal_hot` at @5/@8).

Carrier-scope principle (T.28d): at a `TEAM_WIDE` apex, stat packs + event/role-gated riders (echo, lifesteal, shields, ramp, splash, sustain) apply team-wide -- that IS the apex trade. **Signature/identity effects** (CC-immunity, hexproof opener/pierce) and **carrier-movement** (kiting/backline) stay carrier-only -- trait-guarded (`cc_immunity(trait=...)`, `pierce_hexproof(trait=...)`, `hexproof_opener(trait=...)`) or omitted from the apex, mirroring the `on_death_spawn` trait-guard.

Deferred to **T.31 (D.20)**: Primordial @1 signature mechanics + @3 tier-up.

3.13.8 Roster (post-T.28a rebalance, B.9)

- Kinship pools (sum 60): Beast 14, Spirit 11, Skyborn 9, Scaled 9, Tidekin 9, Swarm 8. One **Tier-10 anchor per kinship**: Mournhollow->Beast, Aurion->Spirit, Aerion->Skyborn, Umbra->Scaled, Nerei->Tidekin, Borealis->Swarm.
- Calling pools (sum ~87): Guardian 9; Hunter/Mystic/Bruiser/Skirmisher/Packmate 8; Stalker/Channeler 7; Warden/Trickster/Mender/Primordial 6.
- `CALLING_TAGS` dropped the 4 dead T.5 tags (Bulwark/Drifter/Harbinger/Emissary) and added **Packmate** (8 cheap T1-3 secondary carriers). Hunter spread toward lower tiers (e.g. Dusk

Bat T2). Primordial shop access is augment-gated (T.31); trait factories ship ready-but-dormant.

3.13.9 Descriptions (T.41b) -- `game/traits/meta.py` + `game/describe.py`

Player-facing trait text flows through the shared description render-layer:

- `traits/meta.py::TRAIT_META: dict[str, TraitMeta]` -- authored `**name + blurb`
 - per-breakpoint `text**` for all **25** traits (transcribed from `trait_catalog.md`, reconciled to the `code`'s counts -- a few drifted during T.28b/c balance: **Bruiser** apex @10 not @8, **Stalker** apex @8 not @7). Each trait's rungs keys == **its factory() breakpoint counts** (V.79, test-guarded).
- `describe.render_trait(id) -> RenderedTrait(name, blurb, rungs)` -- each `RenderedRung(count, text, stat_line)` pairs the authored effect text with a **stat line derived from `_packs.TRAIT_STAT_PACKS`** (the rung's muls/adds, never re-typed -> can't drift, V.79). Pure, no Flet/mutation (V.80).
- **Consumer:** `ui/components/trait_synergies.py::trait_synergies_panel` tooltips (Prep and Combat) -- trait blurb + every breakpoint (• cleared / o) with its **scope** (`[carriers]` = tag-sharers only / `[team]` = whole team, from the rung's `TraitScope`), stat line, and effect text.

3.13.10 Invariants

- V.21 (unique-id count, RNG-free, replay-stable), V.22 (every `Champion.traits` tag resolves in `TRAIT_REGISTRY`; ≥ 1 Kinship + ≥ 1 Calling; Primordial at T10), V.37 (apex/dynamic-threshold + primitive determinism), **V.40 (hexproof targeting -- single-target acquisition excludes `HEXPROOF`; AoE still hits; `pierces_hexproof` bypass; B.15)**, **V.41 (cumulative rungs -- a higher rung re-includes every lower rung's mechanic; sole exception carrier-movement at a `TEAM` apex; B.16)**, **V.79 (every trait has `TRAIT_META` with one description per `factory()` breakpoint; stat line derived from `TRAIT_STAT_PACKS`; T.41b)**. Guarded by `tests/game/test_traits.py` + `tests/game/test_describe.py` + `tests/game/test_hexproof.py` + `content.py` self-asserts.

3.14 Items

Status: LIVING -- must match `src/game/items/`. Audited by `/check`. **Scope:** pointer -- item content lives in code + the system doc. **Reconciled:** 2026-06-16 (T.29a-d).

[done] **SHIPPED (T.29a-d)** -- the item engine is built: 8 components + 16 core + 20 combined + 6 emblems = **50** in `ITEM_REGISTRY`, plus **5** special run-actions in `RUN_ACTION_REGISTRY`. Mana primitive (V.48) + multi-slot (V.49) landed in T.29c/d.

This content doc is a thin pointer; the real material lives in:

- **How the engine works** -> [docs/live/systems/items.md](#) (package layout, components, recipes, equip, reward drops, mana items).
- **The data** -> `src/game/items/` (`base.py`, `combined.py`, `emblems.py`, `special.py`, `recipes.py`); `ids/counts` are `/check`-verified against `ITEM_REGISTRY` / `RUN_ACTION_REGISTRY`.
- **As-designed lore + intent** (frozen) -> `docs/design/content/item_catalog.md`.
- **Build rationale** (frozen) -> `docs/design/tasks/t29_item_engine_plan.md`.

3.14.1 Descriptions (T.41a) -- `game/items/meta.py` + `game/describe.py`

Player-facing item text flows through the shared description render-layer:

- `items/meta.py::ITEM_META: dict[str, ItemMeta]` -- authored **name** + **blurb** for all 50 `ITEM_REGISTRY` ids (transcribed from the frozen `item_catalog.md`). `set(ITEM_META) == set(ITEM_REGISTRY)` is test-guarded (V.78). The blurb is *effect/flavor prose only* -- no stat numbers.
- `describe.render_item(id) -> RenderedEntry(name, text, stat_line)` -- the **stat line is derived by introspecting the item's EffectBundle** (the registered factory built with a null owner; Modifier `mul->+12% STR`, `add->flat`, `crit/pen%->percentage`). The number shown is exactly the number combat applies and cannot drift (V.78); rendering has no side effect (V.80).
- **Consumer:** the Prep item chips (`ui/views/prep.py::_item_chip`) show the rendered name + a tooltip (name - stat line -- blurb - kind/action hint), replacing the old Title-case `_item_label` stopgap. (Trait descriptions: T.41b.)

3.15 Augments -- run-long modifiers

Status: **LIVING** -- audited by `/check`. **Reconciled:** 2026-06-22 (T.31 landed). **Scope:** augment model, registry, offer/reroll, quality curve, the `RunModifiers` combat seam, quest trackers. Code: [src/game/augments.py](#). Design: [docs/design/tasks/t31_augment_system_plan.md](#), [docs/design/content/augment_catalog.md](#).

3.15.1 Model

`Augment(id, name, scope, quality, handler, piece_filter, quest_tracker, blurb)` -- frozen dataclass, registered via `@register_augment(...)` into `AUGMENT_REGISTRY` (`registries.py`). Importing `src.game.augments` populates it (mirrors abilities).

- **AugmentScope** = `TEAM` | `PIECE` | `RUN` -- selects the **handler signature**:
 - `TEAM`: `handler(team: list[Piece], state: dict) -> EffectBundle` -- modifiers/statuses applied to **each** team piece; hooks subscribed **once** (global, close over `team`).
 - `PIECE`: `handler(piece: Piece, state: dict) -> EffectBundle` -- applied to each team piece passing `piece_filter`; hooks close over that piece.
 - `RUN`: `handler(run: Run) -> None` -- mutates `Run` once at **pick time**, no bundle.
 - `state` is `RunModifiers.augment_state` -- read by run-scaling augments (`the_uprising`) and Crest bonuses.
- **AugmentQuality** = `COMMON` | `RARE` | `EPIC` | `PRISMATIC`.
- **54 registered** (~50 catalog across 13/13/16/12 by quality + 3 Primordial-unlock `RUN` augments). Counts: `TEAM` 30 - `RUN` 19 - `PIECE` 5.

`Modifier.source_id` is `augment:<id>` (V.45). All handlers RNG-free (V.2/V.14). Stat magnitudes are **MVP mul values** (catalog ships concepts only) -- a tuning surface (D.11).

3.15.2 Combat seam (V.2 amendment, V.18)

`resolve_combat(..., run_mods=None)` / `resolve_boss_combat(..., run_mods=None)` thread a `RunModifiers(augments, augment_state, run=None)`. `None =>` every non-augment caller (all balance sims) is **byte-for-byte identical** -- guarded by `test_augments.py::test_none_run_mods_byte_identical`.

`compile_loadout(..., run_mods=None)` applies augments in the documented order ([loadout.py](#)):

- **step 3** -- trait resolution reads `augment_state["trait_bonus"]` (Crest/Crown/Worldroot inject virtual carriers into `_resolve_traits` before breakpoint selection).
- **step 3.5** -- flags `piece._has_active_synergy` (the `built_different` filter).
- **step 6** -- `apply_run_augments` builds TEAM/PIECE bundles fresh (V.18: never persisted).
- **step 9** -- `wire_quest_trackers` subscribes quest trackers (no-op unless `run_mods.run` set).

3.15.3 Offers, reroll, quality curve

- `generate_augment_offer(run_seed, node_index, stage_index, *, rerolled=False, exclude=())` -- deterministic 1-of-3 via `augment_seed` (`CH_AUGMENT/CH_REROLL`). Rolls a quality by the stage curve, then a uniform unpicked augment of that quality. No dups; excludes active.
- **Prismatic gated to stage ≥ 2** (D3).
- `quality_weights_for_stage(i)` -- per-stage Common->Prismatic weights (`_STAGE_WEIGHTS`, §5 curve; tuning surface). Prismatic 0 at stage 1, non-decreasing after.
- `apply_augment(run, augment)` -- appends id to `run.active_augments`; RUN handlers fire; quest augments seed `augment_state`.

3.15.4 Quest trackers (§9.3)

`QUEST_TRACKER_REGISTRY + QUEST_TRACKER_EVENTS` (`register_quest_tracker`). Run-level bus subscribers that survive across combats, mutating persistent Run state. MVP quests: `scouts_pay`, `prospector`, `stormbound_trail`, `bloodless_victory`, `the_uprising`, `the_long_hunt`.

3.15.5 Known simplifications (flagged, follow-ups)

- **Living World** (`living_world`) -- **redesigned weather-driven** (away from the boss-only catalog concept, which was inert on 44/50 nodes). Each live weather grants a bespoke team boon echoing its @10 affinity: CLEAR=regen+STR/INT, CLOUDY=-18% incoming, MIST=hexproof opener, RAIN=mana-regen+lifesteal, SNOW=2 `slow` stacks on enemies, THUNDER=AS+lightning-on-hit. Works every fight. (SNOW dogfoods the B.25/V.53 fix -- `slow` now throttles meter advancement.)
- **The Long Hunt** keys on **boss-id kills**, not a `boss_phase2` victim tag (none exists yet) (D5).
- **Primordial Bond** grants a stat tier + once-per-combat barrier; the @2-breakpoint-for-free and @3 fixpoint tier-up (D.20) are **deferred** (D8).
- **Trail Rations / Adrenal Glands / Heart of the Storm** -- modelled as steady stat/regen buffs rather than exact catalog mechanics (D1).
- `salvage_rights` sets `augment_state["salvage_bonus"]`; the sell path (`economy.py`, T.22) must read it.

Chapter 4

AI-Agent Collaboration

Tempest Fauna Trail was built as a deliberate **vibe-coding case study**: almost all of the code was written by an AI coding agent (Claude Code) driven by a human operator, under a process designed to keep that collaboration honest. The project treats the *how we worked with the agent* as a first-class artifact -- every non-trivial milestone carries a mandatory **"Process notes (AI collaboration)"** section and a **prompting-strategy reflection** in its journal entry, precisely because that signal is invisible in the diff and unrecoverable later.

This chapter synthesizes those ~30 journaled reflections into the durable lessons. It is the part of the documentation that no amount of reading the source can reconstruct: where the agent drifted, what the process caught, and how the way of driving the agent evolved over the eight-week build.

4.1 The working model

The collaboration settled into a repeatable shape early and held:

The human steers design at the decision forks; the agent runs the mechanical build. The operator resolves genuine forks up front, then lets the agent execute against a plan -- "resolve the design forks, then let the agent run" is described across the later journals as *the* dominant pattern on this project.

Concretely this is enforced by a spec-driven workflow (detailed in the next chapter): the agent writes a plan doc, the human approves it, the spec is amended, then the build is a comparatively low-entropy transcription of an already-settled design. The recurring refrain -- "*spend the tokens in planning where a mistake is cheap to fix, and the build stops being where surprises live*" -- is the thesis of the whole method, and the journals repeatedly record it paying off.

4.2 The recurring lessons

4.2.1 1. Design docs lie -- verify every primitive against the code

The single most-repeated lesson. Design documents and even the spec contain **illustrative-but-wrong** examples, and the agent will confidently emit spec-consistent, code-inconsistent prose. A partial catalogue of what verification-against-code caught before it shipped:

- `ability_power` / `mana_max` / `attack_damage` -- stat keys that **do not exist** (the real key is intelligence; mana is per-ActiveSlot, not a Piece stat).
- `resolve_combat(..., run_mods=None)` documented as current -- but `run_mods` was an **unbuilt future task** (T.31); the real signature had no such argument.
- Hook event names guessed (`on_attack`) rather than grepped (`on_attack_landed`).
- `CombatOutcome.TEAM_WIN` / `BattleResult.winner` in test assertions -- the real values were `CombatOutcome.WIN` and `BattleResult.outcome`.
- `ft.BarChart` mandated by the spec, but **Flet 0.85 removed the chart widgets from core** -- the framework's own API had drifted.

The generalized rule the project adopted: **writing and verifying are different jobs**. For any doc or handler that *describes* or *reads from* code, a verification-against-code pass must be its own explicit, budgeted step -- never assumed to have happened during authoring. The lesson was eventually extended even to *test assertion targets* (enum values, dataclass field names), which are easy to skip because they "obviously" match the interface you just wrote.

4.2.2 2. Self-review is a separate, load-bearing step -- green tests are not "done"

Repeatedly, an explicit adversarial review pass -- prompted by the operator as "*review the plan for weak spots*" or "*review the changes so far*" -- caught real defects that a **passing test suite did not**:

- A combat-view HP bar that would freeze through ability bursts (caught at *plan* time, before implementation -- described as the single highest-leverage moment of that session).
- A `milli_AS` value that shipped desynced under weather (weather scaled `attack_speed` but not its tiebreak twin).
- A save-load path that would leak a raw `ValueError` past the typed-error contract.

The distilled principle: "*tests passing is not 'done'; an explicit adversarial self-review is a separate step and it keeps earning its keep.*"

4.2.3 3. Distrust the measurement -- a metric reading zero is a claim about the metric

Two of the sharpest bugs were **instrumentation lies**, not system bugs:

- A roster sim reported **0 casts** in a 12,000-tick fight. The first instinct ("pre-existing, out of scope") was wrong; the operator pushed "*question your sim.*" Instrumenting the actual code path showed casts *were* firing -- the recorder's `_on_cast` was a silent no-op, so every cast was invisible.
- Identical mirror-match cells reading `!= 50%` win rate exposed a genuine engine **side-bias** in the tiebreaker, affecting every prior combat result.

Rule adopted: when a metric shows a suspicious extreme (0 / all / exactly 50%), **wrap the function and instrument the path -- do not grep the rendered output**. A number that "won't move" under a lever is often a *mixed* metric hiding two problems; slice it one dimension finer before blaming the lever.

4.2.4 4. Every bug becomes an invariant -- the backprop reflex

The project's most distinctive discipline: a bug fix is not complete until the team asks *"what invariant would have caught this?"* and, where possible, adds one that turns the failure into a **red test**. This "backprop" reflex converted one-off patches into durable guards throughout -- a NaN-weather averaging bug became an invariant; a dead-stat balance hole became a CI axis<->scaling guard that *immediately* flagged two more dead-stat units a hand-audit had missed; a mana-per-slot duplication became a slot-count-invariance guard. The invariants double as **agent guardrails**: they encode, in an executable form, the exact drifts a future agent would otherwise re-introduce.

4.2.5 5. The human catches the hacks and redirects the method

The operator's interventions were consistently high-leverage, and the journals are candid that the human was often right against the agent's in-flight approach:

- *"This seems more like a hack... one logic that handles single/null/multi"* -- forced a materially cleaner multi-slot architecture than the agent's planned **secondary= kward**.
- *"Blast is big, blast is big"* -- vetoed the agent's twice-proposed scope shortcuts in favour of the full, correct refactor.
- *"Just run normal sims, compare win-rates across roles"* -- beat the agent's synthetic-harness instinct; the right experiment design emerged from the operator's domain corrections, not the agent's first plan.
- *"If the issue is developing blind, can you change that?"* -- flipped a self-imposed constraint (see below).

The emergent rule: **when the user smells a hack, stop and re-derive the seam -- do not defend the in-flight approach**. The recommendations the agent surfaced existed largely so they could be *cheaply overridden*; the operator overrode roughly half, which is the point of making forks legible.

4.2.6 6. Ask after a cheap investigation, not instead of one

AskUserQuestion forks worked best when the agent **verified a premise first**, then posed the decision with a recommendation and the concrete trade-off. Twice, feeding the user a code fact *mid-question* flipped their answer (a "does this already work?" turned into a real bug fix once the agent showed the helper didn't filter the gate; a "build it now" reversed to "defer" once the agent showed the content already existed). The pattern: **verify -> state the fact -> re-pose the fork**. For the rote decisions (dozens of invented stat magnitudes), a **decide-and-flag** approach kept momentum -- default the value, log it as a numbered flag for later review, rather than blocking on each one.

4.2.7 7. Headless UI: the operator is the visual oracle -- until it isn't

The agent cannot see rendered Flet output, so early UI work treated the human's screenshots as the only test oracle. Mitigations that worked: a **fake Page harness** to catch import/attribute/exception errors, and leaning on pure, testable playback models for the parts that *are* testable. But the most important lesson was to **not bake a constraint into the recommendation before checking it is real** -- when pushed, the agent found it *could* self-verify via `flet run -w` plus Playwright screenshots, which flipped the entire interaction model. The subtler traps were environmental: **web != desktop** for background-thread->UI safety (a repaint that renders in the web build

silently no-ops on desktop), and a stale **service-worker** frame that nearly sent the agent chasing a phantom bug.

4.3 How the collaboration evolved

The journals show a clear trajectory across the eight weeks:

- **Early entries** are build- and bug-post-mortems written *after* the work.
- **Mid-project**, the discipline of writing process notes for a *plan* (before any code) began surfacing drifts -- an axis-count discrepancy, a dead-field audit -- as first-class findings rather than incidental discoveries during a later build. Plan-time journaling proved worth keeping.
- **Later**, the dominant mode became **conversation-as-design-tool, then skill-chain to execute**: entire subsystems emerged from a handful of "what if X worked like Y?" turns, were locked into the plan and spec, and only then built. The skill chain (`/plan -> /spec -> /build -> /check`, re-entering `/spec` to correct a falsified claim) held its seams -- routing even mid-build spec corrections through the sole mutator kept the encoding and numbering discipline intact and left an honest trail.

Two organizational lessons round it out. **Scope is escalated, not half-built**: when a task's real size exceeded its estimate, it was split along a real seam (traits became T.28 a/b/c/d, each shipping green) rather than left perpetually "80% done." And **concurrency demands blast-radius discipline**: a two-agent parallel experiment succeeded only because the *shared test baseline* -- not just file ownership -- was made explicit, and a careless `git add -A` in a shared worktree once swept another stream's spec edits into the wrong commit (producing a duplicate invariant), yielding the standing rule to **stage your own paths explicitly, never -A, in a shared tree**.

The through-line: the process exists to compensate for the agent's specific failure modes -- confident-but-unverified prose, trust in green tests, trust in its own measurements -- by making verification, self-review, and human design authority into explicit, load-bearing steps rather than assumed ones.

Chapter 5

How the Project Was Built

Tempest Fauna Trail is a FH Technikum Wien project -- two students, eight weeks -- graded on UI, data loading, visualization, code structure, and documentation. What follows is how it was actually built: the **spec-driven method** that governed every change, and the **implementation arc** that method produced, from data models to a playable run loop.

5.1 Spec-Driven Development

The organizing idea is that a single **contract** -- [SPEC.md](#) -- drives the work, and every other document has a defined and different relationship to the truth. Nothing lands without passing through the contract.

5.1.1 The document layers and the LIVING vs FROZEN rule

The rule that prevents documentation drift is the distinction between **LIVING** and **FROZEN** docs:

Layer	Doc	Role	Currency
Contract	SPEC.md	the full contract (§G/§C/§I/§V/§T/§B/§D)	LIVING
Map	ARCHITECTURE.md	where things live and how they interact	LIVING
Reference	docs/live/**	how each subsystem works <i>now</i>	LIVING
Record	docs/design/**	how a task was <i>planned/built</i>	FROZEN
History	docs/journal/**	<i>why</i> , chronologically	FROZEN

LIVING docs describe how things work *now* and **must match the code** -- drift is treated as a bug, the same as a failing test. **FROZEN** docs are point-in-time records that are *never* retro-edited; a frozen task plan is read as a dated snapshot, not current truth. The recurring, expensive failure the project kept paying down was reading a frozen plan as if it were live -- so the discipline is: when a change lands, reconcile the matching LIVING doc in the same commit, and leave the frozen plan alone.

5.1.2 The skill chain

The workflow is mechanized as a chain of skills, each with a single, bounded job:

- **/plan** -- writes a docs/design/tasks/tN*_plan.md before any build; runs a required-reading pass and verifies every primitive against the code; ends in the exact spec deltas needed. Writes only the plan doc.
- **/spec** -- the **sole mutator** of SPEC.md. New rows, invariants, bug backprop. Keeping one mutator preserves the numbering and encoding discipline.
- **/build** -- plan-then-execute against a \$T task; flips task status; auto-runs backprop on a test failure.
- **/check** -- a read-only drift audit: every backticked code reference in the LIVING docs must resolve, every invariant must hold, content counts must match. It reports; it never edits.

The seams held in practice: each stage's output is the next stage's contract, so a mistake gets caught at the cheapest layer. When a build *falsified* a plan's claim (for instance, a change that was assumed byte-identical turned out to shift tick timing), the correction was routed back through **/spec** rather than buried -- the build is allowed to falsify the plan, and the contract is corrected in the open.

5.1.3 Invariants as executable guardrails

The \$V invariants are the backbone. They are not aspirational prose -- most are **CI-guarded**, and each new one is typically born from a bug via *backprop* (the "what invariant would have caught this?" reflex from the previous chapter). A sampling of the load-bearing ones:

- **V.1** -- **game/** has zero Flet imports (pure logic).
- **V.2** -- **resolve_combat** is a pure function: identical inputs yield byte-identical output. No RNG, no clock, no globals.
- **V.14 / V.19** -- the simulation layer imports only pure game logic; all procedural generation derives from (**run_seed**, **node_index**, **channel**).
- **V.3 / V.4** -- the API key is never logged; all HTTP runs on a worker thread.
- Content and balance guards (id resolution, axis<->scaling coverage, weather-metric NaN handling) that turn "silently wrong content" into red tests.

5.2 The implementation arc

Development proceeded **engine-first, UI-last** -- a direct consequence of the purity invariants. Because **game/** is Flet-free and I/O-free, the entire game is testable and simulatable without a UI or a network, so the whole engine, content, and balance layers could be built and validated through the playtest CLI and the power simulation long before any view existed. The tasks (\$T) fall into a few waves:

5.2.1 Foundation (the deterministic core)

τ.1 data models -> τ.2 the two decoupled weather systems (Weather Favor + Affinity Clash) -> τ.3 the tick-based combat engine -> τ.4 the 50-city / 6-stage route -> τ.5 the champion/enemy rosters -> τ.6/τ.7 the OpenWeather client, cache, and 3-stream refresher -> τ.18 the power/scaling formula. This wave established the determinism doctrine that everything downstream relies on.

5.2.2 Framework and tooling

τ.19 seed-deterministic encounter generation -> τ.20 the ability / passive / status framework (typed event bus, hooks, modifiers, registries) -- the declarative substrate all content plugs into -> τ.21 challenge and 2-phase boss encounters with map effects -> τ.24 role-aware enemy formation -> τ.26 unification of combat onto a single tick loop (the old parallel engine was deleted). Alongside, τ.25 the power simulation (matchup sweeps, win-rate/power metrics) and τ.27 the playtest CLI gave the project its two ways to **drive the game headlessly** -- the tools that made engine-first development trustworthy.

5.2.3 Content deepening and balance

The largest wave, and the one where the spec-driven method earned the most. Traits were built in a deliberate split -- τ.28a (framework + declarative stat packs) -> τ.28b (engine primitives) -> τ.28c (mechanic/apex riders via hook idioms) -> τ.28d (hexproof correctness + fold-ins). Items followed the same shape: τ.29-pre reworked the combat **stat substrate** (weather-as-modifiers, `attack_speed` to float), then τ.29a-τ.29d delivered the item engine, the full catalog, the **mana primitive**, and multi-slot pieces. τ.30 implemented all 120 roster ability handlers plus 6 boss kits; τ.31 the augment system; τ.32 the role/intent axis rework; τ.33a/b the scaling classes and a fair total order; τ.34a-c and τ.35a/b the ability-description/tooltip layer and scaling uniformity; and τ.36a-c the roster axis-distribution rebalance, closed with power sweeps. Much of this wave's value was in **design conversation and balance analysis** rather than code -- for example, the INT-ability damage coefficient was tuned empirically *and* cross-checked against a closed-form DPS-parity equilibrium (~ 3.7), so the balance pass ended with math and simulation agreeing, and produced a reusable coefficient rule for future kits.

5.2.4 The player-facing UI (the largest remaining build)

With a complete engine, the view layer was built last: τ.8 theme/components -> τ.9 menu -> τ.10 run-start -> τ.11 Trail view + Canvas route map -> τ.12a-d the combat view (a forward replay stepper, animations, boss support, autoplay) -> τ.13 the Canvas run-summary chart -> τ.14 save/load -> τ.15a/b the reward step and full routing -> τ.23a/b the Prep view (economy, shop, board placement, item equip) -> τ.37a-c the replay backend that combat playback rides on -> τ.38 node-type rewards and survivable "Hearts" -> τ.39 persistent live node weather -> τ.40 the Prep UX overhaul -> τ.41a/b the description render layer. This wave depended on a hard constraint -- the agent cannot see rendered output -- which shaped the collaboration lessons of the previous chapter (visual gating, self-screenshotting, web-vs-desktop threading traps).

5.3 Determinism as the spine

If one property holds the whole project together, it is **determinism**. Because `resolve_combat` is pure and all generation is seeded, the same seed yields the same route, squads, shop, economy, and byte-identical combat -- which is exactly what makes the simulation layer a trustworthy balance oracle and the combat replay possible (state for a view is *recomputed* from the seed, not recorded). Every engine or content change was argued for byte-identity or an explicit, sanctioned **re-baseline**, and the distinction between "byte-identical" and "deterministic but re-baselined" was tracked carefully in the spec. This is the invariant that lets a two-person, eight-week, agent-built project stay coherent: the contract says what must hold, the invariants make it checkable, and the seed makes it reproducible.